

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления

Кафедра информационных технологий автоматизированных систем

К защите допустить:

Заведующий кафедрой ИТАС

_____ В. А. Рыбак

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к дипломному проекту

на тему

**АВТОМАТИЗИРОВАННАЯ ИНФОРМАЦИОННАЯ СИСТЕМА
МУЗЫКАЛЬНОГО СЕРВИСА**

БГУИР ДП 1-53 01 02 039 ПЗ

Студент

И. Г. Жарский

Руководитель

В. И. Ярмолик

Консультанты:

от кафедры ИТАС

В. И. Ярмолик

по экономической части

Е. Н. Макеева

Нормоконтролер

О. Ю. Нехлебова

Рецензент

Минск 2026

Решением рабочей комиссии
допущен(а) к защите дипломного проекта
Председатель рабочей комиссии

_____ (_____)

Подпись

Инициалы и фамилия

« ____ » июня 2026 г.

РЕФЕРАТ

ИНФОРМАЦИОННАЯ СИСТЕМА ПОТОКОВОГО ВЕЩАНИЯ И АНАЛИЗА МУЗЫКАЛЬНОГО КОНТЕНТА : дипломный проект / И. Г. Жарский – Минск : БГУИР, 2026. – п.з. – 91 с., чертежей (плакатов) – 3 л. формата А1.

В дипломном проекте решена задача разработки информационной системы для решения комплекса задач стриминга, глобального поиска и каталогизации медиаданных в реальном времени. Выполнен анализ структуры данных, ее системы управления и существующей информационной системы. Предложены решения для компьютеризации ряда задач, ранее решавшихся вручную или с использованием простейших средств локального хранения, в частности, задачи оптимизированной выборки данных и динамического анализа характеристик аудио. Разработаны алгоритмы пошаговой трансляции медиаданных, выполнено описание информационных потоков, спроектирована база данных, разработаны процедуры контроля входных данных.

Разработано программное обеспечение для решения указанных задач с использованием *Java (Spring Boot)*, *PostgreSQL* и *MinIO S3*. Разработанная информационная система предназначена для использования специалистами мультимедийных платформ и конечными слушателями. Для информационной системы предусмотрены режимы эксплуатации указанными категориями пользователей, а также режим администратора. Подготовлено руководство пользователя и руководство администратора. Составлен и описан набор контрольных примеров. Выполнен расчет ожидаемого экономического эффекта от внедрения системы.

БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ ИНФОРМАТИКИ И
РАДИОЭЛЕКТРОНИКИ

Кафедра информационных технологий автоматизированных систем
(наименование кафедры)

УТВЕРЖДАЮ
Заведующий кафедрой
В. А. Рыбак
« 06 » февраля 2026 г.

ЗАДАНИЕ
на дипломный проект

Обучающемуся Жарскому Ивану Геннадьевичу
(фамилия, собственное имя, отчество (если таковое имеется))

Курс 4 Учебная группа 220602

Специальность 1-53 01 02 Специализация

Тема дипломного проекта: Автоматизированная информационная система музыкального сервиса

Утверждена приказом ректора 06.02.2026 г. № 249-с

Исходные данные к дипломному проекту: Тип операционной системы – ОС Windows
сервер базы данных – PostgreSQL; языки программирования JavaScript, Java; MinIO

Перечень подлежащих разработке вопросов или краткое содержание расчетно-
пояснительной записки:

Задачи, решаемые в проекте: автоматизация процессов глобального поиска медиаконтента,
динамического анализа акустических характеристик аудиофайлов и их оптимизированной
поточковой трансляции чанками по запросу пользователя

Содержание пояснительной записки:

Введение

1. Описание и анализ предметной области.

2. Проектирование структуры информационной системы.

3. Программная реализация музыкального сервиса.

4. Технико-экономическое обоснование эффективности разработки автоматизированной
информационной системы музыкального сервиса

Заключение

Список использованных источников

Приложение А (обязательное) Листинг метода *streamTrack*

Перечень графического материала (с точным указанием обязательных чертежей и графиков):
Диаграмма вариантов использования музыкального сервиса (ПЛ) – формат А1, лист 1.

Схема базы данных музыкального сервиса (ПЛ) – формат А1, лист 1.

Консультанты по дипломному проекту (с указанием разделов, по которым они консультируют): консультант по проекту – ст. преподаватель В.И. Ярмолик
консультант по экономической части – ст. преподаватель кафедры экономики Е. Н. Макеева

Примерный календарный график выполнения дипломного проекта:

Анализ предметной области. Изучение методов экономического прогнозирования и процедур ценообразования. Расчет экономической эффективности - 23.03.2026 г.

Проектирование структуры музыкального сервиса. Разработка структуры информационного обеспечения. Разработка основных алгоритмов музыкального сервиса. Разработка структуры программного обеспечения - 27.04.2026 г.

Разработка автоматизированной информационной системы.

Отладка информационной системы. Разработка контрольных примеров. Решение задач эргономического обеспечения. Подготовка чернового варианта пояснительной записки и графического материала - 18.05.2026 г.

Оформление графического материала и пояснительной записки. Подготовка доклада и презентации для защиты дипломного проекта - 25.05.2026 г.

Дата выдачи задания 06.02.2026 г.

Срок сдачи законченного дипломного проекта 01.06.2026 г.

Руководитель дипломного проекта

(подпись)

В. И. Ярмолик

(инициалы, фамилия)

Подпись обучающегося

Дата 06.02.2026 г.

СОДЕРЖАНИЕ

Введение.....	6
1 Описание и анализ предметной области.....	8
1.1 Структура системы управления и виды деятельности.....	8
1.2 Информационные системы	10
1.3 Анализ аналогичных информационных систем	12
1.4 Постановка задачи дипломного проекта	22
2 Проектирование структуры информационной системы	24
2.1 Организационно-экономическая сущность задачи.....	24
2.2 Структура информационной системы.....	26
2.3 Математическое и алгоритмическое обеспечение.....	27
2.4 Информационное обеспечение	30
2.5 Техническое и системное программное обеспечение	41
2.6 Эргономическое обеспечение	43
2.7 Организационное обеспечение	45
3 Программная реализация информационной системы.....	48
3.1 Выбор программных средств реализации ИС.....	48
3.2 Структура программного обеспечения ИС.....	49
3.3 Разработка программного кода ИС.....	52
3.4 Руководство пользователя	57
3.5 Описание интерфейса	60
4 Экономическое обоснование разработки и реализации на рынке автоматизированной информационной системы музыкального сервиса....	61
4.1 Характеристика музыкального сервиса, разрабатываемого для реализации на рынке.....	61
4.2 Расчет инвестиций в разработку музыкального сервиса для его реализации на рынке.....	62
4.3 Расчет экономического эффекта от реализации программного средства на рынке.....	66
4.4 Расчет показателей экономической эффективности разработки и реализации программного средства на рынке.....	68
4.5 Вывод по результатам расчета	68
Заключение.....	70
Список использованных источников.....	71
Приложение А (обязательное) Листинг кода.....	72
Ведомость дипломного проекта.....	92

ВВЕДЕНИЕ

В современных условиях индустрия цифрового контента переживает стремительную трансформацию. Музыкальные стриминговые платформы стали основным каналом распространения медиапродукции, однако большая часть рынка сосредоточена вокруг глобальных сервисов, которые не всегда отвечают узкоспециализированным требованиям независимых лейблов и нишевых медиаплатформ.

Основной проблемой в данной предметной области является отсутствие автоматизированных инструментов, позволяющих эффективно управлять медиабibliothekой, анализировать предпочтения аудитории и обеспечивать безопасное хранение контента с учетом специфики независимого сектора. Применение современных информационных технологий, таких как объектные хранилища (*S3*), системы управления базами данных с поддержкой сложных аналитических запросов (*PostgreSQL*) и алгоритмы машинного обучения для рекомендательных систем, позволяет существенно повысить качество взаимодействия с пользователем и оптимизировать внутренние процессы управления контентом.

Целью данного дипломного проекта является проектирование и разработка информационной системы для автоматизации процессов управления медиабibliothekой и формирования персонализированных рекомендаций для слушателей независимых музыкальных платформ.

Для достижения поставленной цели необходимо решить следующие задачи:

- 1 Провести анализ существующих решений и выявить основные функциональные требования к проектируемой системе.
- 2 Спроектировать архитектуру информационной системы. Схему базы данных, обеспечивающую целостность и масштабируемость.
- 3 Разработать программный комплекс, включающий серверную логику (*backend*) для обработки медиаданных и реализации рекомендательных алгоритмов.
- 4 Реализовать удобный пользовательский интерфейс, отвечающий современным стандартам эргономики и юзабилити.
- 5 Обеспечить безопасность хранения и передачи данных, а также корректную работу с медиаресурсами.

В работе применяются методы системного анализа, объектно-ориентированного проектирования и моделирования бизнес-процессов (*DFD*).

Для разработки системы используются следующие программные и технические средства:

- *Java/Node.js* для построения серверной части.
- *PostgreSQL* (с использованием *JSONB* для гибкого хранения метаданных).
- *Docker*, обеспечивающий переносимость и стабильность программной среды.
- *S3*-совместимые объектные хранилища для медиаконтента.
- *RESTful API* для взаимодействия клиента и сервера.

Дипломный проект выполнен самостоятельно, проверен в системе «Антиплагиат». Процент оригинальности соответствует норме, установленной кафедрой. Цитирования обозначены ссылками на публикации, указанные в «Списке использованных источников»

Дипломный проект выполнен самостоятельно, проверен в системе «Антиплагиат». Процент оригинальности соответствует норме, установленной кафедрой. Цитирования обозначены ссылками на публикации, указанные в «Списке использованных источников».

1 ОПИСАНИЕ И АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Структура системы управления и виды деятельности

БГУИР сегодня - это крупный учебно-научно-инновационный комплекс, в структуру которого входят 8 факультетов: компьютерного проектирования (ФКП), информационных технологий и управления (ФИТУ), радиотехники и электроники (ФРЭ), компьютерных систем и сетей (ФКСиС), информационной безопасности (ФИБ), инженерно-экономический (ИЭФ), военный факультет (ВФ), факультет доуниверситетской подготовки и профессиональной ориентации (ФДППО) [1].

Университет включает в себя 9 факультетов, 32 кафедры, институт информационных технологий. Филиалом университета является Минский радиотехнический колледж. БГУИР осуществляет подготовку студентов по 41 специальности первой ступени высшего образования, по 15 - второй ступени, по 30 – аспирантуры и 15 – докторантуры.

На балансе университета находится 8 учебно-практических корпусов и 5 общежитий. При этом они не формируют кампуса, так как расположены в нескольких местах небольшими группами. Большинство зданий университета построены в 1960-х — 1980-х годах в стиле советского архитектурного модернизма.

Основные виды деятельности БГУИР:

1 Образовательная деятельность — подготовка высококвалифицированных ИТ-специалистов, инженеров по радиоэлектронике, микроэлектронике и информационной безопасности на всех уровнях: бакалавриат, магистратура и аспирантура.

2 Научная и инновационная деятельность — проведение фундаментальных и прикладных исследований в области искусственного интеллекта, кибербезопасности, нанотехнологий и радиотехники, а также внедрение разработок в реальный сектор экономики.

3 Международное сотрудничество — активное взаимодействие с ведущими зарубежными техническими вузами и мировыми ИТ-гигантами, реализация программ «двойных дипломов» и участие в международных образовательных и научных грантах.

4 Взаимодействие с индустрией и бизнес-образование — создание совместных лабораторий с крупнейшими ИТ-компаниями, проведение сертифицированных курсов (*Cisco*, *SAP*, *Microsoft* и др.) и подготовка кадров под конкретные задачи цифровой экономики.

5 Цифровая трансформация и просвещение — проведение профильных конференций, хакатонов и олимпиад по программированию, направленных на популяризацию инженерного образования и развитие цифровых компетенций в обществе.

6 Техническая экспертиза и консалтинг — предоставление услуг по разработке программного обеспечения, проектированию радиоэлектронных систем, а также проведение экспертиз в области защиты информации.

БГУИР обладает развитой многоуровневой структурой и эффективной системой управления, адаптированной под задачи высокотехнологичного вуза. Структура университета объединяет факультеты, общеуниверситетские кафедры, научно-исследовательские части, отраслевые лаборатории, центры трансфера технологий, библиотеку, издательство и мощный центр информационных технологий.

Во главе БГУИР стоит ректор, который осуществляет общее руководство стратегическим развитием и всеми направлениями деятельности вуза. Команда проректоров курирует ключевые блоки: учебную работу (организация учебных планов, аттестация), научную деятельность (инновационные разработки, аспирантура) и воспитательную работу. Деканы факультетов управляют отдельными подразделениями, отвечая за качество подготовки инженеров, подбор преподавательского состава и связь с профильными предприятиями-заказчиками кадров.

Кафедры являются базовыми звеньями, обеспечивающими проведение занятий по техническим и гуманитарным дисциплинам. Они также отвечают за работу совместных учебных лабораторий с ИТ-компаниями, выполнение научно-исследовательских и опытно-конструкторских работ (НИОКР) и актуализацию учебных программ под требования рынка. В университете активно действует система студенческого самоуправления, представленная студенческим советом, профкомом и молодежными центрами, которые организуют хакатоны, технические конкурсы, культурные фестивали и волонтерские проекты.

Фундаментальным элементом системы управления БГУИР является его цифровая экосистема. Она включает в себя передовую информационную инфраструктуру: систему «Электронный университет», личные кабинеты студентов и преподавателей, электронные ведомости, репозиторий научных трудов и платформы дистанционного обучения. Это позволяет автоматизировать большинство управленческих процессов и обеспечить прозрачность контроля успеваемости.

Одной из главных задач системы управления БГУИР является создание инновационной образовательной среды, стимулирующей научно-техническое

творчество, обеспечение лидерства в сфере цифровых технологий, а также укрепление престижа университета как ведущего центра подготовки кадров для «экономики знаний».

Прохождение преддипломной практики осуществлялось в отделе автоматизированных систем управления и информационных технологий. Основные направления деятельности отдела [2]:

- 1 Совершенствование процесса управления университетом и обучения на основе применения автоматизированных систем управления (АСУ). Использования современных средств телекоммуникации и информационных технологий.

- 2 Внедрение и использование АСУ в управлении образовательным процессом.

- 3 Организация электронного документооборота структурных подразделений университета.

- 4 Разработка и внедрение в образовательный процесс новых образовательных информационных технологий.

- 5 Разработка сопровождающего программного обеспечения для электронных учебно-методических комплексов.

- 6 Разработка и администрирование тестирующих программ для контроля знаний.

1.2 Информационные системы

БГУИР активно использует различные информационные системы в своей деятельности, направленные на автоматизацию управления и учебного процесса.

Одной из ключевых составляющих цифровой экосистемы БГУИР является Интегрированная информационная система (ИИС). Это уникальная разработка университета, объединяющая в себе функции управления учебным процессом, личного кабинета студента и преподавателя, а также инструменты электронного документооборота.

ИИС БГУИР представляет собой единое цифровое пространство, которое автоматизирует взаимодействие всех участников образовательного процесса. Основные возможности системы включают:

- 1 Личный кабинет студента. Здесь доступно актуальное электронное расписание, зачетная книжка с оценками за все семестры, а также информация о назначенных стипендиях и пропусках занятий.

2 Личный кабинет преподавателя. Позволяет вести электронные журналы, выставлять оценки, контролировать посещаемость и загружать учебно-методические комплексы.

3 Управление учебными планами. Система хранит актуальные программы обучения по всем специальностям, обеспечивая прозрачность образовательной траектории.

4 Заказ справок и документов. Студенты могут удаленно запрашивать справки об обучении, что значительно сокращает бюрократическую нагрузку.

Для реализации дистанционных образовательных технологий в БГУИР используется Система электронного обучения (СЭО). Она служит основным инструментом для взаимодействия студентов и преподавателей вне учебных аудиторий.

Система электронного обучения (СЭО) – комплекс программных и технических средств, обеспечивающих реализацию образовательного процесса с использованием ДОТ, мониторинг и протоколирование хода и результатов изучения обучающимся учебных дисциплин (образовательной программы).

Электронный образовательный ресурс по учебной дисциплине (ЭОР) – это представленные в электронном виде и размещенные в СЭО систематизированные учебные, научные и методические материалы (или ссылки на них) по учебной дисциплине для самостоятельного изучения обучающимся теоретического материала и выполнения им видов учебной деятельности, предусмотренных учебным планом (программой) в зависимости от формы получения образования, набор фондов оценочных средств для диагностики сформированных компетенций.

Модуль электронного образовательного ресурса – структурная единица ЭОР, содержащая логически завершенную часть учебного материала по учебной дисциплине, которая включает раздел (разделы), одну или несколько тем. Модуль может не содержать материалов для практического выполнения обучающимся видов учебной деятельности, предусмотренных программой, в зависимости от формы получения образования. ЭОР содержит не менее 2 модулей (как правило, 3), а в случае осуществления подготовки по учебной дисциплине в дистанционной форме получения образования – не менее 3.

Практическая часть разрабатывается по каждой форме обучения на основании учебно-программной документации. Наименование и количество элементов практической части (обязательных видов учебной деятельности обучающегося) определяется учебными планами для соответствующей формы получения образования. Содержание видов учебной деятельности обучающегося определяется учебной программой учебной дисциплины

(учебно-методических карт для соответствующей формы обучения). В зависимости от формы получения образования практическая часть может включать следующие элементы:

1 Практические и семинарские занятия (ПЗ). Могут включать в себя перечень рефератов, варианты заданий (задач), примеры их решений, «кейсы», методические материалы, учебные и справочные издания (ссылки на учебные и справочные издания, находящиеся в открытом доступе в сети Интернет), для подготовки и выполнения ПЗ и так далее.

2 Лабораторные работы (ЛР). ЛР могут включать в себя варианты заданий, алгоритм и примеры выполнения, методические рекомендации, учебные и справочные издания (ссылки на учебные и справочные издания, находящиеся в библиотеке БГУИР или открытом доступе в сети Интернет), требования к оформлению отчета и так далее.

3 Индивидуальные практические работы (ИПР). ИПР включают в себя варианты заданий, примеры их выполнения, методические рекомендации, учебные и справочные издания (ссылки на учебные и справочные издания, находящиеся в открытом доступе в сети Интернет), требования к оформлению и так далее.

4 Контрольные работы (КР). КР включают в себя варианты заданий, примеры их выполнения, методические рекомендации, учебные и справочные издания (ссылки на учебные и справочные издания, находящиеся в библиотеке БГУИР или открытом доступе в сети Интернет) и так далее.

5 Типовые расчеты, расчетные, расчетно-графические работы, рефераты. Могут включать в себя варианты заданий, примеры их выполнения, методические рекомендации, учебные и справочные издания (ссылки на учебные и справочные издания, находящиеся в библиотеке БГУИР или в открытом доступе в сети Интернет) требования к оформлению и так далее.

Структурные элементы модулей ЭОР разрабатываются в форматах, поддерживаемых СЭО (веб-страницы, документы форматов *.docx, *.pdf, *.pptx, изображения, встроенные форматы СЭО и другие).

1.3 Анализ аналогичных информационных систем

Современные музыкальные сервисы давно перестали быть просто «библиотеками файлов» с кнопкой воспроизведения. Сегодня это сложнейшие экосистемы, работа которых строится на стыке облачных технологий, обработки больших данных и психологии потребления контента.

Фундаментальный сдвиг в индустрии произошел, когда модель «владения» (покупка дисков или MP3-файлов) сменилась моделью «доступа».

Пользователь теперь платит не за конкретный альбом, а за право в любой момент времени обратиться к облачному хранилищу, содержащему практически всю когда-либо записанную музыку. Это превращает сервис из простого посредника в полноценную инфраструктуру, которая должна работать без сбоев на миллионах устройств одновременно.

С технической точки зрения, современная АИС (Автоматизированные информационные системы) музыкального стриминга — это распределенная система с колоссальной нагрузкой. Главный вызов здесь заключается в доставке контента. Музыкальные файлы весят немало, а пользователь ожидает мгновенного старта воспроизведения. Для этого сервисы используют технологию адаптивного битрейта: система на лету анализирует качество интернет-соединения и незаметно для слушателя переключает качество аудиопотока, чтобы музыка не прерывалась. Сами данные хранятся не на одном сервере, а распределены по сети доставки контента (*CDN*), что позволяет отдавать файл из ближайшей к пользователю географической точки.

Внутренняя логика таких систем пропитана микроотчетностью. В отличие от продажи физического носителя, где сделка разовая, стриминг подразумевает непрерывный подсчет. Каждое прослушивание трека дольше 30 секунд фиксируется системой как событие. Эти миллиарды событий ежедневно обрабатываются для формирования отчетности перед правообладателями и расчета роялти. Это требует огромных мощностей для работы с *Big Data*, так как система должна не просто хранить историю, но и агрегировать её в реальном времени.

Сегодняшний музыкальный сервис — это прежде всего рекомендательный движок. Из-за избытка контента (ежедневно на платформы загружаются десятки тысяч новых треков) пользователь физически не может выбирать музыку сам. Поэтому центр тяжести в разработке АИС сместился в сторону машинного обучения. Системы анализируют не только жанр или исполнителя, но и «цифровой отпечаток» самого звука — темп, тональность, плотность инструментов. Более того, алгоритмы учитывают контекст: время суток, геолокацию и даже тип устройства, на котором запущено приложение, чтобы предложить музыку, максимально подходящую под текущую ситуацию.

Музыкальные платформы стали частью социального пространства. Возможность делиться плейлистами, смотреть, что слушают друзья в реальном времени, или участвовать в коллективном прослушивании превращает сервис в специфическую социальную сеть. Для артистов же такие системы стали основным инструментом аналитики: они видят демографию своей аудитории, популярность конкретных треков в разных регионах и могут

планировать гастроли, опираясь на жесткие цифры из панели управления сервисом.

Теперь проанализируем аналогичные системы: *Spotify* и Яндекс Музыка.

Spotify в современной ИТ-индустрии считается золотым стандартом распределенных систем, который превратил прослушивание музыки из линейного процесса в высокотехнологичный сервис на основе данных. В отличие от традиционных плееров, архитектура этого сервиса построена на принципах максимальной автономности и масштабируемости, что позволяет ему обслуживать сотни миллионов пользователей без видимых задержек.

Техническое устройство *Spotify* базируется на идее разделения монолитного приложения на сотни независимых микросервисов. Каждый такой сервис отвечает за строго определенную операцию: от обработки поискового запроса до генерации обложки плейлиста. Это позволяет системе быть невероятно отказоустойчивой — если у сервиса ломается модуль социальных связей, пользователь все равно сможет слушать музыку. В дипломе такую структуру можно описать как модульную архитектуру, где каждый компонент взаимодействует с другими через стандартизированные программные интерфейсы (*API*), обеспечивая гибкость всей системы.

Одной из главных инженерных побед *Spotify* стала разработка проприетарных протоколов передачи данных. Чтобы музыка начинала играть мгновенно после нажатия кнопки, сервис использует агрессивное кэширование и предсказание поведения. Приложение анализирует, какой трек в плейлисте идет следующим, и начинает подгружать его начало в фоновом режиме еще до того, как текущая песня закончится. На системном уровне это реализуется через сложную иерархию серверов: популярные треки хранятся «ближе» к пользователю в оперативной памяти граничных серверов, тогда как редкие записи лежат в глубоких облачных хранилищах.

Сердце *Spotify* — это его рекомендательная модель, которая фактически является самой сложнейшей системой обработки сигналов и анализа текстов. Сервис не просто группирует музыку по жанрам, он создает многомерное векторное пространство, где каждый трек имеет сотни характеристик: от акустичности и танцевальности до «энергии» звука. Система постоянно сопоставляет профили прослушивания разных людей, находя неочевидные закономерности. Если тысячи пользователей с похожими вкусами внезапно начали слушать нового исполнителя, алгоритм автоматически поднимет его приоритет в твоих персональных подборках. Для АИС это означает наличие мощного аналитического модуля, работающего с нейронными сетями в реальном времени.

Spotify генерирует колоссальный объем технических событий в секунду. Каждое нажатие на паузу, пропуск трека или изменение громкости отправляется в централизованную систему сбора данных. Эти логи используются не только для статистики, но и для обучения моделей машинного обучения. Внутренняя инфраструктура сервиса спроектирована так, чтобы переваривать эти терабайты данных, превращая сырые логи в осмысленные отчеты для лейблов и персональные итоги года для слушателей. В контексте дипломной работы это можно представить как систему событийного мониторинга, которая связывает активность пользователя с бизнес-логикой сервиса.

Уникальной чертой системы является функция *Connect*, которая превращает все устройства пользователя в единую управляемую сеть. Это реализуется через поддержку постоянного веб-сокета соединения между клиентом и сервером. Когда ты переключаешь трек на компьютере с помощью телефона, сигнал проходит через облачный координатор, который мгновенно обновляет состояние плеера на всех активных сессиях. Такая бесшовная интеграция требует сложной логики синхронизации состояний в базе данных, чтобы избежать конфликтов при одновременном доступе с разных устройств.

Интерфейс *Spotify* — это эталон акцентного минимализма и карточной архитектуры. В 2026 году он окончательно закрепил за собой статус «умного черного зеркала», где дизайн не отвлекает от контента, а подстраивается под него.

Дизайн интерфейса *Spotify* представляет собой современную многопанельную структуру, построенную на принципах глубокой темной темы, где визуальный контент преобладает над текстовым, как представлено на рисунке 1.1. Основное пространство разделено на функциональные зоны, которые обеспечивают бесшовную навигацию без необходимости перезагрузки страниц. Левая узкая панель служит быстрым доступом к личной медиатеке, используя компактные квадратные пиктограммы обложек, что позволяет экономить рабочее место. Центральная область отдана под динамическую сетку рекомендаций и недавних прослушиваний, где крупные плитки с мягким скруглением углов и контрастным текстом позволяют пользователю мгновенно ориентироваться в контенте.

Особое внимание уделено правой информационной панели, которая акцентирует внимание на текущем треке через полноразмерное изображение обложки альбома, создавая глубокое визуальное погружение. Нижняя часть интерфейса выделена под фиксированную панель управления воспроизведением, которая визуально отделена от основного контента. Здесь используются интуитивно понятные иконки управления, шкала прогресса и

дополнительные инструменты, такие как управление очередью и громкостью. Цветовая палитра базируется на сочетании радикально черного и темно-серого фонов, на которых ярко выделяются акцентные элементы — например, зеленый цвет активных кнопок или индикаторов воспроизведения. Такой подход минимизирует нагрузку на зрение при длительном использовании и выводит на первый план художественное оформление музыкальных релизов, превращая интерфейс в эстетичное дополнение к аудиопотоку.

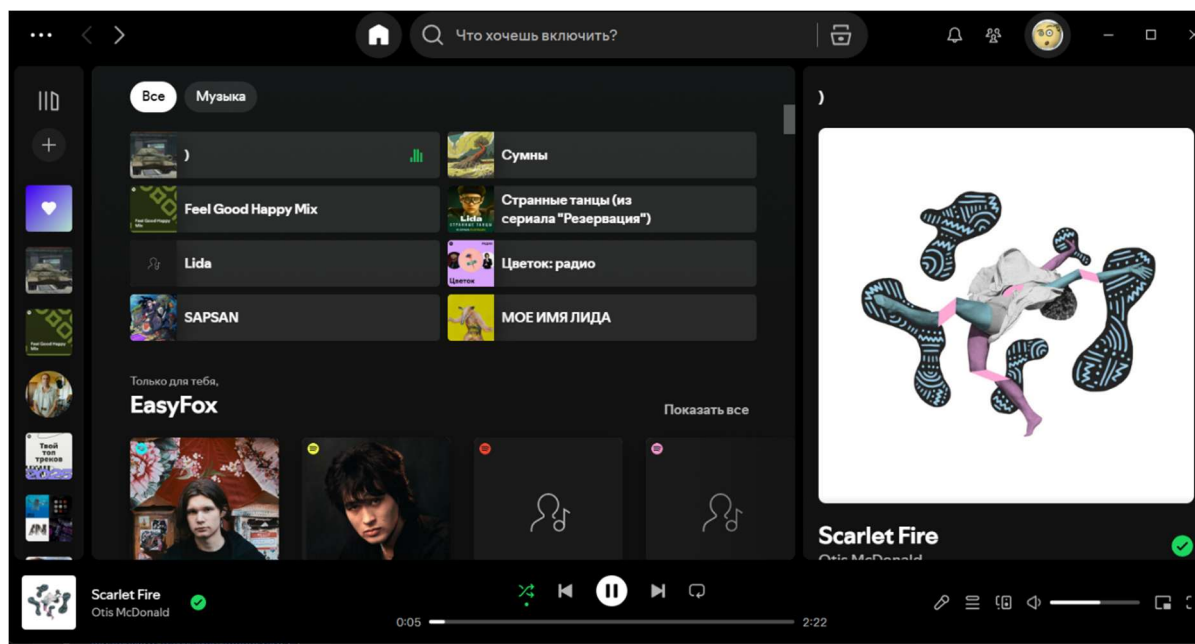


Рисунок 1.1 – Главная страница *Spotify*

Интерфейс мобильного приложения *Spotify*, представленный на рисунке 1.2, демонстрирует современную концепцию вертикально-ориентированного адаптивного дизайна, где ключевым приоритетом является удобство управления одной рукой. Визуальная иерархия выстроена таким образом, что наиболее актуальный контент располагается в верхней трети экрана, обеспечивая мгновенный визуальный контакт с последними действиями пользователя. Цветовое решение базируется на глубоком темном фоне, что позволяет эффективно использовать контраст для выделения интерактивных элементов, таких как ярко-зеленая кнопка фильтрации контента, служащая главным цветовым акцентом верхней панели.

Центральная часть экрана организована в виде сетки из восьми компактных карточек с закругленными углами, которые объединяют графический контент и текстовые метаданные в единый навигационный блок. Такой подход минимизирует когнитивную нагрузку, позволяя пользователю считывать информацию через узнаваемые обложки артистов и плейлистов.

Ниже располагается секция персональных рекомендаций, использующая более крупные квадратные плитки с градиентной заливкой, что визуальнo отделяет алгоритмически созданный контент от истории прослушиваний и акцентирует внимание на функциях социального взаимодействия, таких как создание общих плейлистов.

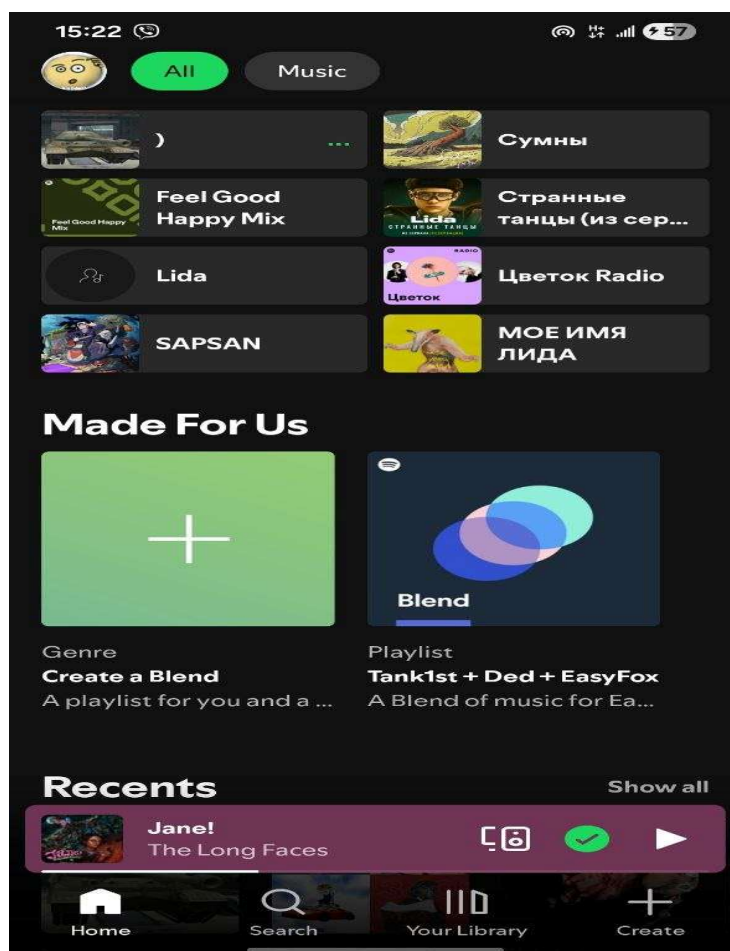


Рисунок 1.2 – Главная страница мобильного *Spotify*

Особое значение в архитектуре интерфейса имеет нижняя навигационная панель и плавающий мини-плеер. Плеер визуальнo выделен мягким вишневым оттенком, извлеченным из палитры обложки текущего трека, что создает эффект динамического интерфейса, подстраивающегося под контекст прослушивания. Панель управления в самом низу экрана обеспечивает быстрый доступ к ключевым разделам системы — поиску, библиотеке и созданию контента — через минималистичные контурные иконки. Общая композиция элементов стремится к максимальной информативности при сохранении «воздуха» в интерфейсе, что характерно для

современных АИС мультимедийной направленности, где дизайн служит лишь проводником к основному контенту.

Яндекс Музыка — это пример того, как классический музыкальный сервис трансформировался в продукт, полностью управляемый искусственным интеллектом. Если *Spotify* делает ставку на социальные связи и кураторские плейлисты, то Яндекс пошел по пути максимальной автоматизации и «бесшовного» потребления контента внутри своей экосистемы.

Ключевое отличие Яндекса заключается в смещении фокуса с библиотеки (где пользователь ищет альбом) на «поток» (где система сама решает, что играть). Центральным элементом АИС здесь является система «Моя волна». С точки зрения системного анализа, это бесконечный генератор очереди воспроизведения, который пересчитывает приоритеты выдачи треков после каждого действия пользователя: лайка, пропуска или даже изменения громкости. Это делает систему не статичным хранилищем, а динамическим процессом.

Технологический стек Яндекса опирается на две мощные группы алгоритмов, работающих в связке:

- 1 Коллаборативная фильтрация. Система анализирует миллиарды взаимодействий миллионов пользователей. Она ищет «цифровых близнецов» — людей с идентичными паттернами поведения — и предлагает треки, которые зашли им, но еще не прослушаны вами.

- 2 Глубокий анализ аудиоконтента (*Deep Audio Analysis*): Нейросети Яндекса «слушают» спектрограмму каждого трека. Они выделяют тембр голоса, бит, ритм и даже эмоциональный окрас (настроение). Это позволяет системе подмешивать в поток абсолютно новых, неизвестных артистов только потому, что их звучание математически схоже с тем, что любит пользователь.

Яндекс Музыка уникальна своей глубокой интеграцией с другими сервисами (Алиса, Навигатор, Яндекс Книги). АИС получает данные о контексте пользователя: если музыка запущена через умную колонку вечером, система отдаст приоритет спокойным трекам; если в машине через Навигатор — динамичным. Технически это реализуется через единый идентификатор (*Yandex ID*) и общую шину данных, которая передает контекстные теги в рекомендательный движок.

Интересной инженерной особенностью является динамическая визуализация (цветовая индикация «Моей волны»). Это не просто анимация, а результат работы алгоритма, который на лету анализирует частотные характеристики проигрываемого аудиопотока и сопоставляет их с метаданными настроения трека.

Внутри системы развернут мощный аналитический блок для правообладателей. Яндекс предоставляет артистам доступ к данным в реальном времени: они видят, как их треки попадают в рекомендации, каков процент дослушиваний и как география слушателей влияет на популярность. С точки зрения АИС — это сложная система ролевого доступа к базам данных *Big Data*, где сырые логи прослушиваний агрегируются в понятные бизнес-метрики.

Интерфейс Яндекс Музыки на представленном рисунке 1.3 демонстрирует современный подход к проектированию стриминговых платформ, где ключевой акцент сделан на функциональном зонировании и алгоритмической подаче контента. Дизайн выполнен в темной цветовой гамме, которая служит нейтральным фоном для ярких интерактивных элементов и обложек, не отвлекая пользователя от основной задачи — выбора музыки.

Левая вертикальная панель представляет собой классическое навигационное меню с интуитивно понятными пиктограммами и текстовыми подписями, что обеспечивает быстрый переход между глобальными разделами сервиса. В нижней части этой панели расположен блок авторизации пользователя и кнопка загрузки десктопного приложения, что подчеркивает экосистемный характер продукта.

Центральное рабочее пространство организовано по принципу горизонтальных лент (каруселей). В верхней части расположены приоритетные блоки «Избранное» и «История», оформленные в виде крупных карточек с мягким градиентом, что выделяет их на общем фоне. Ниже представлен уникальный блок «Моя волна», реализованный через систему тегов-фильтров (по жанру, настроению, активности). Это позволяет пользователю кастомизировать поток воспроизведения в один клик. Визуальное решение этих кнопок выполнено с использованием абстрактных паттернов, что подчеркивает «интеллектуальность» и динамичность алгоритмов.

Нижнюю часть интерфейса занимает фиксированный плеер, который является центром управления звуком. Он визуально отделен от контентной области тонкой разделительной линией и содержит все необходимые инструменты: индикатор прогресса трека, кнопки управления очередью, лайк и настройки громкости. Яркая желтая кнопка воспроизведения в центре плеера служит главным визуальным якорем всего интерфейса. Дизайн в целом выглядит цельным и сбалансированным, сочетая в себе строгость системных компонентов с яркостью и эмоциональностью музыкального контента.

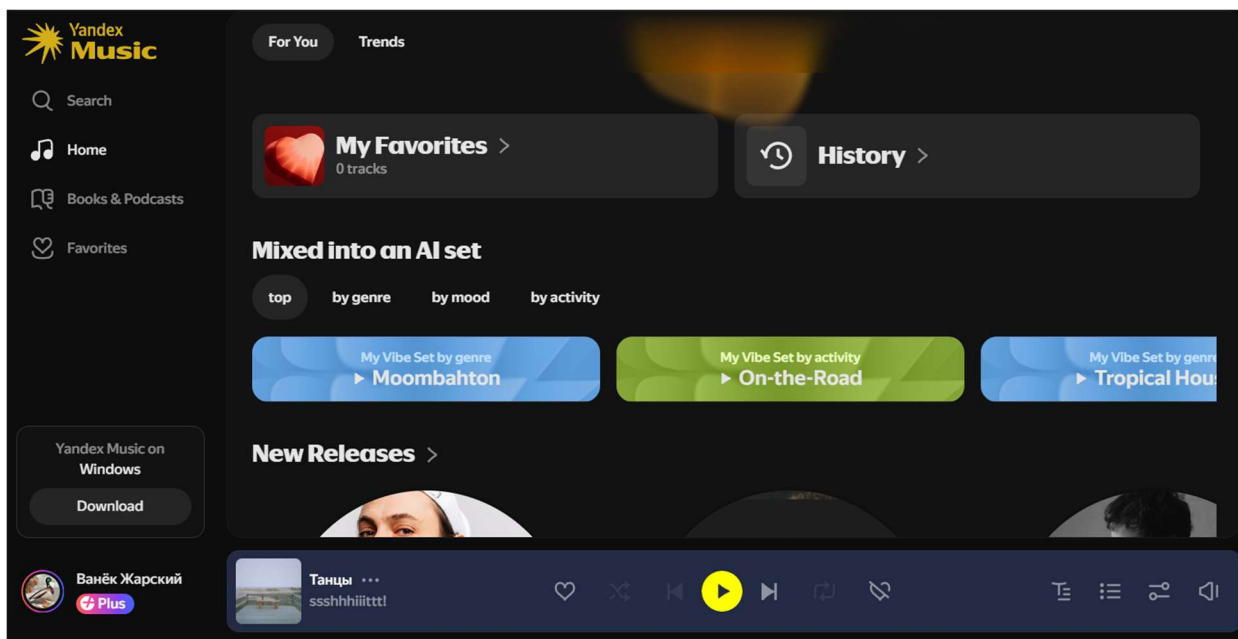


Рисунок 1.3 – Главная страница Яндекс Музыка

YouTube Music — это уникальный гибрид классического стриминга и крупнейшего в мире видеохостинга. Если *Spotify* — это «умный алгоритм», а Яндекс — «бесконечный поток», то *YouTube Music* — это «всеобъемлющий контекст».

Главное техническое преимущество *YouTube Music* — доступ к поисковой истории всей экосистемы *Google*. АИС сервиса знает не только то, что ты слушал, но и что ты искал в *Google* или смотрел на основном *YouTube*. Это позволяет системе строить «сверхпрофиль» пользователя.

В отличие от конкурентов, база данных *YouTube Music* не ограничена только официальными релизами от лейблов.

Система индексирует миллиарды видеороликов. Благодаря алгоритмам компьютерного зрения и анализа звуковых отпечатков (*Content ID*), сервис вычлняет аудиодорожки из видео (каверы, лайвы с концертов, ремиксы, фанатские нарезки) и бесшовно встраивает их в музыкальную библиотеку.

Это пример системы с неструктурированными источниками данных, где механизм фильтрации и классификации (отсеивание плохого звука от студийного) является ключевым бизнес-процессом.

YouTube Music делает ставку на «здесь и сейчас». Используя данные о геолокации и активности пользователя, система меняет интерфейс в реальном времени. Если АИС видит, что пользователь находится в спортзале (по *GPS* и истории посещений), она выводит на главный экран энергичные плейлисты. Если пользователь в аэропорту — подборки для путешествий. Это реализуется

через динамическую генерацию интерфейса на основе контекстных метаданных, что является отличной темой для отдельной главы диплома.

Одной из сложнейших инженерных задач, решенных в *YouTube Music*, является мгновенное переключение между аудиодорожкой и видеоклипом без прерывания звука.

Система синхронизирует два разных потока данных (аудиофайл высокого качества и видеофайл) по таймкодам с точностью до миллисекунды.

Это можно описать как алгоритм синхронизации мультимедийных потоков из гетерогенных источников, что подчеркивает технологическую сложность архитектуры.

Поисковый движок *YouTube Music* — один из самых мощных в индустрии, так как он наследует технологии *Google* Поиска. Пользователь может вбить «песня, где парень поет в пустыне про любовь» или часть невнятного текста, и система найдет трек. Это пример внедрения *NLP* (обработки естественного языка) и семантического поиска в музыкальную АИС, что позволяет находить контент даже при отсутствии точных метаданных.

YouTube Music — это пример того, как музыкальный сервис может использовать внешние данные (видео, поиск, локация) для создания максимально точного пользовательского опыта.

Интерфейс *YouTube* «Музыка», представленный на рисунке 1.4, отражает философию масштабируемости и интеграции, где музыкальный сервис является частью глобального видеохостинга. В отличие от *Spotify* или Яндекс Музыки, здесь дизайн строится на строгой блочной системе, которая должна одинаково эффективно отображать как профессиональные студийные альбомы, так и пользовательский видеоконтент. Визуальная доминанта смещена в сторону максимально крупных превью-карточек, которые занимают почти всё полезное пространство центральной части экрана, создавая эффект витрины.

Левая навигационная панель выполнена в классическом стиле сервисов *Google*: она максимально лаконична и использует тонкие контурные иконки на темном фоне. Это позволяет пользователю быстро переключаться между основным *YouTube* и его музыкальным подсегментом, сохраняя единообразие пользовательского опыта. В верхней части интерфейса находится массивная строка поиска, которая визуально объединена с кнопками голосового ввода и создания контента, что подчеркивает активную роль пользователя в системе — он не только потребитель, но и потенциальный автор.

Особое внимание в дизайне уделено карточкам плейлистов в ленте рекомендаций. Каждая плитка имеет вертикальную ориентацию и включает в

себя яркое изображение с наложенным поверх него контрастным типографическим оформлением названия подборки. В нижней части каждой карточки расположен полупрозрачный индикатор количества треков, что добавляет интерфейсу информативности без перегрузки деталями. Использование радикально черного фона в сочетании с яркими, насыщенными цветами обложек создает динамичную и современную эстетику, которая ориентирована на визуальное восприятие и быстрый скроллинг в поисках нужного настроения [3].

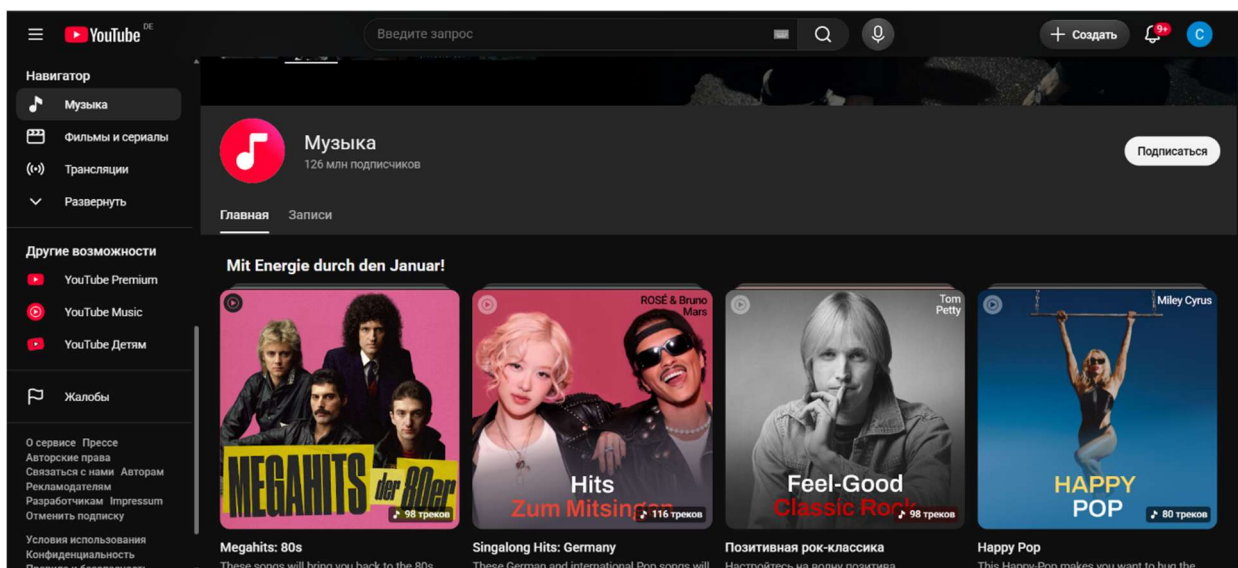


Рисунок 1.4 – Главная страница *YouTube Music*

1.4 Постановка задачи дипломного проекта

В настоящее время многие независимые музыкальные лейблы и современные медиа-платформы не имеют собственных автоматизированных информационных систем, которые бы в полной мере отвечали потребностям анализа предпочтений слушателей и эффективного управления медиатекой. Для решения задач по хранению музыкального контента, формированию плейлистов и взаимодействию с аудиторией в данный момент используются разрозненные облачные хранилища и сторонние социальные сети, не обеспечивающие единой структуры данных [4].

При анализе существующих систем-аналогов (таких как *Spotify* или Яндекс.Музыка) определено, что их готовые решения не могут быть интегрированы в частные инфраструктуры из-за закрытости алгоритмов и отсутствия возможности кастомизации под узкие жанровые или

корпоративные требования, а качество рекомендательных моделей напрямую влияет на удержание аудитории в рамках конкретного сервиса.

Таким образом, определена задача дипломного проекта: необходимо создать автоматизированную информационную систему музыкального сервиса с модулем персональных рекомендаций для управления аудио-каталогом и обеспечения доступа к контенту как для администраторов (кураторов лейбла), так и для конечных слушателей.

Система должна иметь адаптивный веб-интерфейс, созданный с учетом требований к юзабилити современных мультимедийных приложений и специфики потребления аудио-контента.

Кроме того, определены исходные данные к проекту. Система должна корректно работать в таких браузерах как *Google Chrome* (версия 90 и выше), *Mozilla Firefox* (версия 85 и выше), *Safari* (версия 14 и выше), *Microsoft Edge* (версия 90 и выше). Используемые технологии: язык программирования — *Java* для серверной части, *JavaScript (React.js)*, *HTML5*, *CSS3* для клиентской части; система управления базами данных — *PostgreSQL*; хранение медиа-файлов — объектное хранилище (*S3*-совместимое).

2 ПРОЕКТИРОВАНИЕ СТРУКТУРЫ ИНФОРМАЦИОННОЙ СИСТЕМЫ

2.1 Организационно-экономическая сущность задачи

Назначением разрабатываемой АИС является автоматизация процессов управления музыкальным контентом, хранения медиа-активов и персонализированной доставки контента конечным пользователям (слушателям). Система призвана решить проблему отсутствия единой инфраструктуры управления для независимых медиа-платформ, минимизировать трудозатраты на ручную обработку метаданных и обеспечить масштабируемость при увеличении объема медиатеки. Внедрение системы позволяет перейти от разрозненных методов хранения (облачные диски, неструктурированные базы) к централизованному управлению данными, что существенно повышает эффективность деятельности лейбла или стриминговой площадки.

Система реализует следующие основные функции:

- 1 Централизованное управление контентом. Загрузка, редактирование метаданных и классификация аудиофайлов по жанрам, артистам и альбомам.
- 2 Интеллектуальная рекомендательная подсистема. Автоматическое формирование очереди воспроизведения на основе анализа предпочтений пользователя и акустических характеристик треков.
- 3 Пользовательский стриминг. Обеспечение бесперебойной передачи аудиоданных с использованием адаптивного битрейта для минимизации задержек.
- 4 Аналитическая отчетность. Учет статистики прослушиваний для расчета роялти правообладателям и оценки популярности релизов.

Режимы работы ИС включают:

- 1 Оперативный (*Online*) режим. Обработка запросов пользователей в режиме реального времени (воспроизведение, поиск, навигация по каталогу).
- 2 Пакетный (*Batch*) режим. Выполнение регламентных заданий по обновлению рекомендательных моделей, агрегации статистики за отчетные периоды и созданию бэкапов.

В качестве входной информации выступают:

- 1 Данные от администраторов/артистов. Аудиофайлы в формате *lossless/compressed*, метаданные релизов, обложки (графические файлы).
- 2 Данные от слушателей. Параметры профиля, история действий (лайки, пропуски треков, время прослушивания), поисковые запросы.

3 Системные данные. Логи событий (*events*), данные об авторизации и подписках.

Выходная информация включает в себя:

- аудиопотоки для конечных потребителей;
- персонализированные списки рекомендаций и динамические плейлисты;
- аналитические отчеты о популярности контента для правообладателей;
- системные уведомления и статусы транзакций;

Основными подразделениями-источниками входной информации являются «Отдел контента» (загрузка данных) и «Пользовательская база». Получателями выходной информации выступают конечные слушатели (в виде потокового вещания) и администрация сервиса/лейбла (в виде аналитических сводок).

Функции воспроизведения контента, навигации и формирования рекомендаций реализуются в непрерывном режиме (*on-demand*). Задачи по сбору и агрегации статистических данных выполняются в режиме реального времени, в то время как формирование детальных отчетов по выплатам роялти и глубинной аналитике аудитории осуществляется с ежемесячной периодичностью. Обновление рекомендательных моделей нейросетевого уровня производится в рамках ночного окна обслуживания, чтобы исключить влияние на производительность системы для пользователей.

Разрабатываемая АИС взаимодействует с внешними компонентами инфраструктуры:

1 Объектное хранилище (*S3-compatible*). Выступает как внешний источник хранения тяжелого медиа-контента, взаимодействуя с основной ИС через *API*;

2 Система биллинга. Связь с платежными шлюзами для обработки подписок пользователей;

3 База данных (*PostgreSQL*). Централизованное хранилище всех метаданных, к которому обращаются все программные модули системы для синхронизации состояний;

4 Мониторинг. Интеграция с внешними системами логирования для контроля работоспособности сервера в режиме 24/7.

Данная структура ИС обеспечивает гибкость архитектуры, позволяя независимо масштабировать модули хранения контента и модули бизнес-логики, что является критически важным фактором для экономической эффективности проекта.

2.2 Структура информационной системы

Разрабатываемая информационная система спроектирована на основе модульной архитектуры, обеспечивающей независимость компонентов, масштабируемость и простоту поддержки. Структурно систему можно разделить на три функциональных уровня: уровень представления (клиентская часть), уровень бизнес-логики (серверная часть) и уровень данных.

Система включает в себя следующие основные подсистемы и модули: подсистема управления пользователями (*Identity & Access Management*), подсистема управления контентом (*Content Management Module*), подсистема потокового воспроизведения и рекомендаций (*Streaming & Recs Module*), подсистема аналитики и отчетности (*Analytics Module*).

Подсистема управления пользователями отвечает за аутентификацию и авторизацию пользователей. Он реализует функции регистрации, входа в систему, восстановления пароля и управления профилем. Обеспечивает безопасность доступа и разграничение прав (роли: слушатель, администратор, правообладатель). Взаимодействует с базой данных пользователей и предоставляет токены доступа для всех остальных модулей.

Подсистема управления контентом – ядро системы, отвечающее за медиатеку. Этот модуль также включает в себя:

1 Блок загрузки (*Uploader*). Модуль для администраторов, позволяющий загружать аудиофайлы и обложки, автоматически генерировать метаданные.

2 Блок хранения (*Media Handler*). Осуществляет передачу аудиоданных в S3-совместимое объектное хранилище и возвращает ссылки на контент.

3 Каталогизатор. Реализует структуру хранения: Артисты – Альбомы – Треки.

Подсистема потокового воспроизведения и рекомендаций — самая критическая часть системы с точки зрения нагрузки. Состоит из двух модулей:

– Модуль воспроизведения обеспечивает бесшовную передачу аудиопотока клиенту (стриминг).

– Модуль интеллектуальных рекомендаций анализирует историю действий пользователя (лайки, пропуски) и на основе бизнес-логики формирует динамические очереди воспроизведения («Моя волна»).

Структура модуля интеллектуальных рекомендаций представлена на рисунке 2.1. Модуль интеллектуальных рекомендаций выступает центральным компонентом системы, отвечающим за автоматизацию процесса подбора

музыкального контента и обеспечение высокой степени персонализации для каждого слушателя. Процесс функционирования данного модуля начинается с этапа сбора и профилирования данных, в ходе которого осуществляется детальный анализ истории прослушиваний, а также фиксируются предпочтения пользователя, выраженные через взаимодействия с системой, такие как лайки или пропуски треков. Параллельно с обработкой поведенческих факторов система выполняет анализ аудио-контента, включающий в себя систематизацию метаданных и извлечение ключевых акустических признаков, что позволяет глубже понимать природу предлагаемых произведений. Полученная информация становится базой для генерации кандидатов, где на данном этапе применяются алгоритмы коллаборативной и контентной фильтрации, позволяющие сформировать широкий предварительный список потенциально интересных композиций. Завершающим процессом в работе модуля является финальное ранжирование, в рамках которого происходит присвоение весов релевантности и оценка качества рекомендаций, чтобы сформировать итоговую выдачу, максимально соответствующую актуальным интересам пользователя, что в совокупности создает адаптивный механизм управления медиапоток.

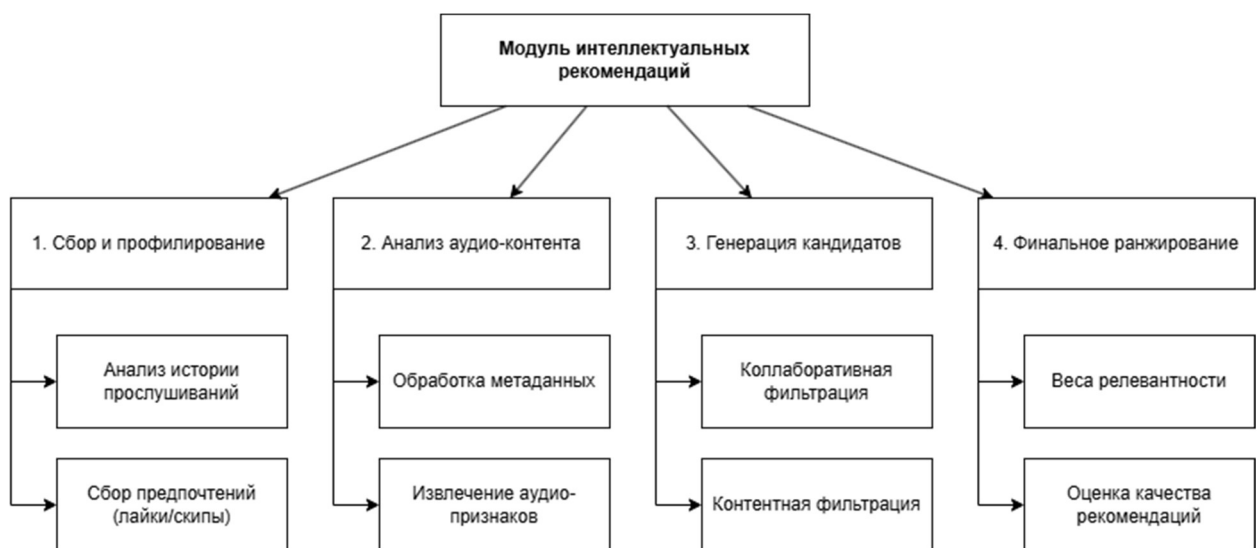


Рисунок 2.1 – Структура модуля интеллектуальных рекомендаций

2.3 Математическое и алгоритмическое обеспечение

В рамках разрабатываемой ИС используются методы обработки данных, направленные на повышение релевантности поиска и персонализацию контента.

Для реализации функции поиска по каталогу используется алгоритм, основанный на метрике схожести строк (например, расстояние Левенштейна или метод взвешенного ранжирования). Стандартный *SQL*-оператор *LIKE* недостаточно эффективен, поэтому применяется модель векторного представления запросов.

Поиск осуществляется по принципу оценки релевантности (*TF-IDF/BM25*) [5]. Каждое слово в названии трека и запросе пользователя преобразуется в вектор, учитывающий частоту встречаемости слов в коллекции (*IDF*). Это позволяет системе находить треки, даже если пользователь допустил опечатку.

Основным методом рекомендаций является *Content-Based Filtering* (фильтрация на основе содержания) с использованием косинусной близости. Для каждого трека i и пользователя u формируется вектор характеристик V (жанр, темп, тональность). Степень сходства между вектором профиля пользователя A и вектором трека B вычисляется по формуле косинусного сходства:

$$similarity = \cos(\varnothing) = \frac{A \cdot B}{\|A\| \cdot \|B\|} = \frac{\sum_{i=1}^n A_i \cdot B_i}{\sqrt{\sum_{i=1}^n A_i^2} \cdot \sqrt{\sum_{i=1}^n B_i^2}} \quad (2.1)$$

где: A_i, B_i – компоненты векторов признаков;

n – размерность пространства признаков (количество параметров контента).

Система ранжирует треки по убыванию значения косинуса, предлагая пользователю наиболее близкие по вектору характеристик произведения.

Для формирования динамических плейлистов «Популярное» используется функция экспоненциального затухания, которая позволяет учитывать свежесть контента:

$$S = \frac{V}{e^{\lambda * t}} \quad (2.2)$$

где V — количество прослушиваний (интегральный показатель);

t — время, прошедшее с момента релиза (в днях);

λ — коэффициент затухания (подбирается экспериментально).

Этот алгоритм гарантирует, что новые треки имеют шанс попасть в чарты наравне со старыми хитами.

Для иллюстрации работы рекомендательного движка приведем логику выбора контента, которая представлена на блок-схеме на рисунке 2.2.

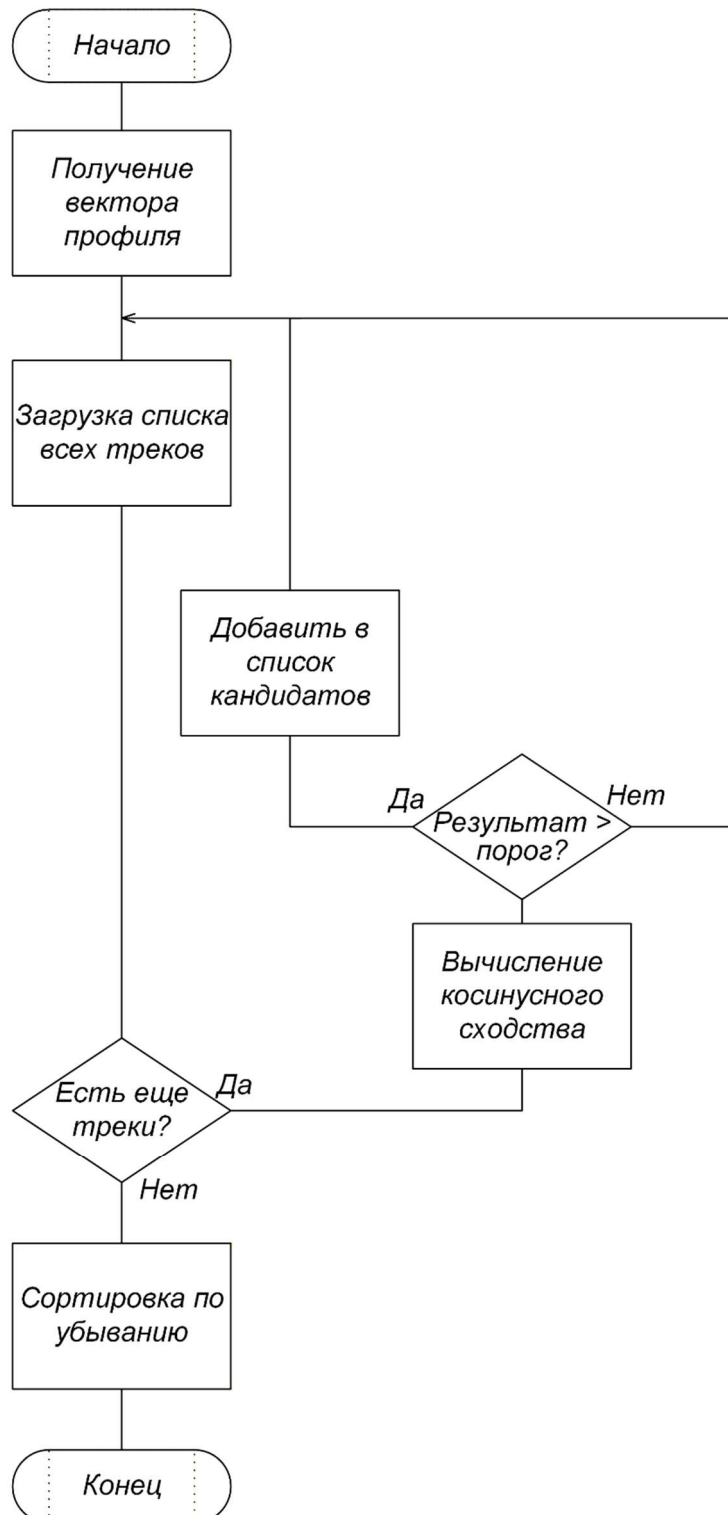


Рисунок 2.2 – Блок-схема алгоритма работы рекомендаций

2.4 Информационное обеспечение

Информационное обеспечение разрабатываемой информационной системы (ИС) представляет собой комплексную структуру, объединяющую методы сбора, хранения, обработки и передачи данных, необходимых для реализации функций музыкальной платформы. Целью информационного обеспечения является создание единого информационного пространства, гарантирующего достоверность, актуальность и целостность данных на всех этапах их жизненного цикла.

В основу организации информационного обеспечения положен процессный подход, согласно которому данные проходят через следующие функциональные блоки:

- 1 Блок сбора (входные данные). Первичная регистрация данных пользователями, администраторами или внешними *API*-сервисами.

- 2 Блок обработки (логический слой). Преобразование первичных данных в структурированную информацию с помощью бизнес-логики и алгоритмов рекомендательного движка.

- 3 Блок хранения (база данных). Обеспечение долговременного сохранения метаданных в реляционной СУБД и медиаконтента в объектном хранилище.

- 4 Блок выдачи (выходные данные). Формирование информационных ответов для фронтенд-части системы (интерфейса).

Блок сбора данных выступает в роли первичного интерфейса взаимодействия информационной системы с внешними субъектами, обеспечивая прием и первичную обработку поступающей информации. Этот компонент является критически важным звеном, так как именно на этом этапе происходит первичная фильтрация и проверка поступающих сведений, от качества которой напрямую зависит целостность всей базы данных в дальнейшем.

В процессе функционирования платформы блок сбора непрерывно взаимодействует с пользовательскими запросами, принимая как явные данные при регистрации и заполнении профиля, так и сигналы, возникающие при навигации, поисковых запросах или взаимодействии с аудиоплеером. Валидация на этом этапе носит строгий характер, так как она призвана защитить систему от некорректных форматов ввода и потенциальных угроз безопасности, включая попытки несанкционированного доступа или внедрения вредоносного кода.

Параллельно с этим функционирует подсистема приема контента, ориентированная на артистов и представителей лейблов. При загрузке

медиафайлов происходит разделение входящего потока на два независимых канала: метаданные поступают в основной контур системы для быстрой индексации и последующего поиска, а тяжелые бинарные аудиофайлы направляются через специализированный шлюз непосредственно в объектное хранилище. Такой подход исключает риск перегрузки основного сервера обработки данных и обеспечивает корректную работу с большими объемами мультимедийного контента.

Немаловажную роль в блоке сбора играет механизм телеметрии, который фиксирует системные события, включая логи работы программных интерфейсов и служебную информацию о действиях пользователей. Все поступающие данные, вне зависимости от их источника, проходят процедуру нормализации, которая приводит их к единому стандартизированному виду, пригодному для дальнейшей обработки алгоритмами рекомендательного движка. Завершается работа блока передачей отфильтрованных и структурированных данных в очередь сообщений или непосредственно в базу данных, что гарантирует асинхронную обработку и высокую отказоустойчивость системы при возникновении пиковых нагрузок.

Блок обработки представляет собой центральное ядро информационной системы, выполняющее функции интеллектуального анализа, преобразования и маршрутизации данных. Его основная роль заключается в трансформации первичной, зачастую слабоструктурированной информации, поступившей из блока сбора, в формализованные сущности, готовые для хранения в базе данных или передачи конечному пользователю. На этом этапе реализуется бизнес-логика всей платформы, которая определяет правила поведения системы при возникновении тех или иных событий, регистрация пользователя, загрузка нового трека или запрос на формирование рекомендательной ленты.

Функционально этот блок разделяется на несколько уровней обработки, начиная с базовой проверки бизнес-правил, где происходит контроль прав доступа и валидация прав пользователя на совершение конкретной операции. Далее следует уровень алгоритмической обработки, на котором работает рекомендательный движок. Здесь данные о предпочтениях пользователя сопоставляются с характеристиками музыкального контента посредством вычисления коэффициентов сходства, что позволяет генерировать персонализированные списки воспроизведения. Вся вычислительная нагрузка, связанная с математическими операциями и ранжированием, концентрируется именно в этом блоке, что позволяет разгрузить базу данных и оставить ей только функции долговременного хранения.

Важным аспектом функционирования блока обработки является обеспечение согласованности состояния системы. В случаях, когда процесс

обработки требует выполнения нескольких последовательных действий, блок управления организует их в транзакции, гарантируя, что при сбое на одном из этапов данные не останутся в промежуточном, частично измененном состоянии. Кроме того, данный блок реализует механизм асинхронной обработки фоновых задач, таких как генерация обложек, анализ темпа композиции или перекодирование аудиофайлов, что предотвращает блокировку основного интерфейса и поддерживает высокую отзывчивость платформы даже при выполнении ресурсоемких вычислений.

Завершается работа блока подготовкой данных для вывода, что подразумевает их сериализацию в удобный для фронтенд-приложения формат и, при необходимости, обогащение дополнительной информацией из смежных таблиц базы данных. В результате такого многоуровневого подхода обеспечивается строгое разделение ответственности между компонентами системы, где блок обработки выступает в роли «диспетчера» и «аналитика», гарантируя, что любая информация, попадающая в систему или покидающая её, соответствует установленным стандартам качества и безопасности.

Блок хранения данных представляет собой фундамент информационной системы, обеспечивающий персистентность всей совокупности сведений, накопленных в процессе эксплуатации платформы. Данный компонент спроектирован с учетом специфики музыкального контента, что обусловило выбор гибридной архитектуры хранения, разделяющей потоки метаданных и «тяжелых» бинарных файлов. Этот подход позволяет минимизировать нагрузку на основной сервер баз данных и оптимизировать операции чтения и записи, что критически важно для обеспечения высокой отзывчивости интерфейса при работе с тысячами одновременно воспроизводимых треков.

Центральным элементом блока хранения является реляционная система управления базами данных, отвечающая за структурированное хранение метаданных, таких как учетные записи пользователей, описания артистов, структура альбомов и взаимосвязи внутри плейлистов. Выбор реляционной модели обусловлен необходимостью строгого соблюдения принципов *ACID*, что гарантирует атомарность транзакций и ссылочную целостность данных в любых ситуациях, включая сбои электропитания или сетевые разрывы. В этом слое активно используются индексы и механизмы связей, обеспечивающие высокую скорость выполнения поисковых запросов и операций ранжирования, которые инициируются блоком обработки. База данных в данной архитектуре выступает в роли своеобразного «каталога», содержащего не сами медиафайлы, а ссылки и указатели на их местоположение в системе.

Для размещения аудиоконтента и графических материалов в системе предусмотрено использование объектного хранилища, функционирующего по

принципу S3-сервисов. В отличие от реляционных баз данных, объектное хранилище оптимизировано для работы с неструктурированными данными большого объема, обеспечивая высокую степень масштабируемости и устойчивости к отказам за счет распределенного хранения фрагментов контента. Интеграция между этими двумя уровнями осуществляется посредством хранения в базе данных уникальных идентификаторов и путей (*URI*) к объектам, что позволяет системе эффективно управлять доступом к файлам через пре-сайт ссылки. Такой метод обеспечивает не только безопасность контента, предотвращая прямую несанкционированную загрузку файлов, но и позволяет распределять нагрузку на сетевую инфраструктуру, используя современные технологии доставки контента.

Надежность и сохранность данных в блоке хранения обеспечиваются комплексом механизмов регулярного резервного копирования и репликации. Процедуры бэкапа настроены таким образом, чтобы создавать снимки состояния базы данных с заданным интервалом, при этом объекты в хранилище защищены версионированием, предотвращающим случайное удаление или перезапись файлов. Система мониторинга постоянно отслеживает целостность данных, автоматически выявляя несоответствия и инициируя процессы восстановления в случае необходимости. В конечном счете, архитектура блока хранения спроектирована таким образом, чтобы быть независимой от остальных узлов системы, позволяя при необходимости расширять дисковое пространство или мигрировать на более производительные аппаратные решения без остановки работы всей платформы.

Блок выдачи данных выступает в роли завершающего этапа информационного цикла, выполняя функции трансляции обработанных результатов в структурированный вид, доступный для визуализации в пользовательских интерфейсах или для передачи внешним программным системам. Его архитектурная роль заключается в обеспечении максимально эффективной и безопасной доставки информации конечному потребителю, при этом скрывая сложность внутренней структуры данных и обеспечивая высокую скорость отклика системы. Этот компонент является интерфейсом взаимодействия между серверной логикой и клиентскими приложениями, где закладываются принципы формирования пользовательского опыта и удобства навигации.

Основой работы данного блока является *API*-шлюз, который обрабатывает входящие запросы от веб-версии платформы или мобильного приложения, маршрутизирует их к соответствующим контроллерам и собирает итоговый ответ. Здесь происходит сериализация данных из внутренних

моделей базы данных в стандартизированные форматы, такие как *JSON*, что обеспечивает высокую степень совместимости с различными клиентскими платформами. Важным аспектом на этом этапе является фильтрация информации, при которой пользователю передаются только те поля, которые необходимы для текущей задачи, что позволяет снизить объем передаваемого трафика и сократить время загрузки интерфейса.

Особое внимание в блоке выдачи уделено механизму безопасной доставки медиаконтента, который требует специфических подходов для предотвращения несанкционированного доступа. Вместо прямой передачи ссылок на аудиофайлы, система генерирует временные, криптографически подписанные *URL*-адреса, которые позволяют клиенту получить доступ к контенту в объектном хранилище лишь на ограниченный период времени. Это обеспечивает защиту интеллектуальной собственности правообладателей и одновременно снижает нагрузку на основной сервер, перенаправляя трафик воспроизведения непосредственно на узлы доставки контента (*CDN*).

Для обеспечения высокой производительности и масштабируемости блок выдачи активно использует стратегии кэширования ответов и пагинации данных. Часто запрашиваемая информация, например, списки популярных треков или данные профиля, сохраняется в промежуточном кэше, что позволяет исключить избыточные обращения к базе данных и значительно ускорить ответ системы при пиковых нагрузках. При больших объемах данных, таких как лента рекомендаций или результаты поиска, система отдает информацию порционно, что предотвращает переполнение оперативной памяти клиентского устройства и обеспечивает плавность отображения контента. В конечном итоге, этот блок не только предоставляет данные, но и управляет состоянием клиентской сессии, гарантируя, что пользователь получает актуальную и корректную информацию в режиме реального времени, соответствующую его текущим правам доступа и предпочтениям.

Процесс взаимодействия этих элементов представлен на схеме информационных потоков, представленной на рисунке 2.3.

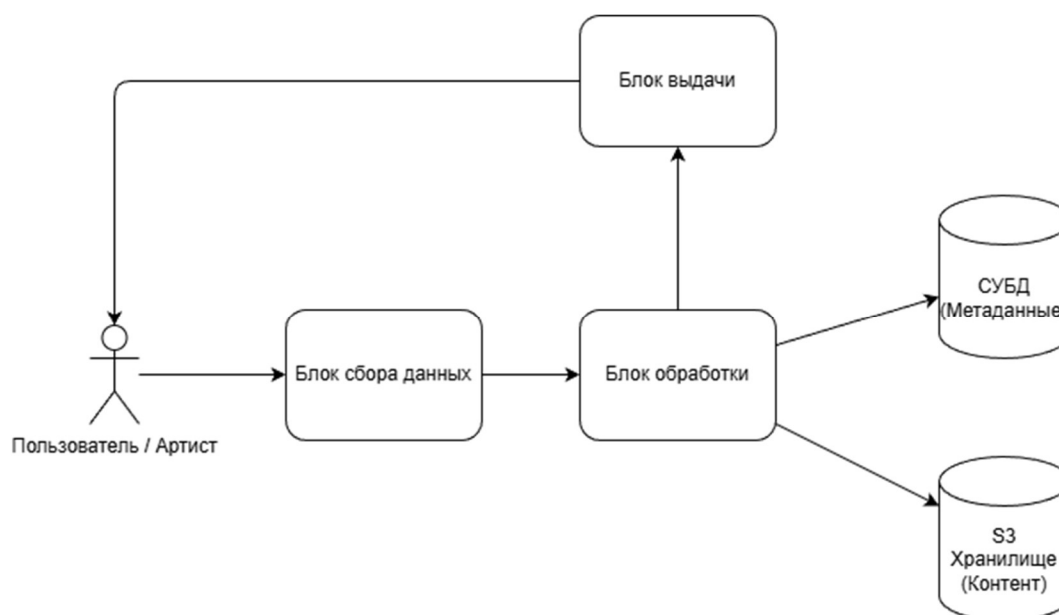


Рисунок 2.3 – Схема информационных потоков

Входная информация разрабатываемой ИС включает в себя сведения, необходимые для инициализации процессов регистрации, поиска, формирования рекомендаций и управления медиаконтентом. Обработка данных осуществляется как в ручном режиме (пользовательский ввод), так и в автоматическом (загрузка файлов, получение данных через *API*).

Для удобства анализа входные данные разделены на две группы: пользовательские операции (ввод через интерфейс) и информационные объекты (загружаемые данные).

Таблица 2.1 – Входные данные процессов взаимодействия пользователей с системой

Этап ввода	Тип ввода	Входные данные
Процесс аутентификации	Ручной ввод	Имя пользователя (логин), Пароль
Процесс поиска контента	Ручной ввод	Поисковый запрос (текстовая строка)
Процесс настройки фильтров	Выбор из списка	Жанр, Темп (<i>BPM</i>), Настроение, Тональность
Процесс управления плейлистами	Ручной ввод	Название плейлиста
Процесс управления плейлистами	Выбор из списка	Статус доступа (публичный/приватный)

Таблица 2.2 – Входные данные (информационные объекты) системы

Наименование	Источник	Получатель	Периодичность	Описание
Аудиофайл (трек)	Артист / Лейбл	Модуль хранения контента	По мере поступления	Бинарный файл (<i>MP3</i> , <i>WAV</i>), поток данных
Метаданные трека	Артист / Лейбл	БД метаданных	По мере поступления	<i>JSON</i> -файл или заполненная форма (название, артист, длительность)
Графический актив	Артист / Лейбл	Модуль хранения контента	По мере поступления	Файл изображения (обложка альбома, логотип)
Логи активности	Система (<i>Telemetry</i>)	Модуль аналитики	В режиме реального времени	Электронный лог-файл (<i>JSON/CSV</i>)

Для обеспечения корректности работы системы все входные данные проходят процедуру первичной валидации:

1 Форматная проверка. Соответствие типов данных (например, ограничение длины строки для пароля, проверка допустимых форматов аудиофайлов).

2 Диапазонная проверка. Контроль корректности числовых значений (например, значения ВРМ должны быть в диапазоне от 40 до 250).

3 Логическая проверка. Проверка уникальности имен пользователей при регистрации и отсутствия дубликатов при загрузке треков с одинаковыми метаданными.

Для работы системы необходимо построить базу данных(БД)[6]. Диаграмма БД представлена на рисунке 2.4. Описание всех таблиц представлено на таблице 2.3. Как видно из таблицы, структура данных разделена на два контура: основные таблицы обеспечивают работу базового функционала платформы, тогда как аналитические таблицы (*track_features*, *user_interactions*, *user_profiles*) служат информационной базой для функционирования алгоритмов персонализированных рекомендаций.

Использование такой структуры позволяет разграничить операционную нагрузку и обеспечить высокую скорость отклика системы при формировании рекомендательной ленты.

Выходная информация системы является результатом обработки входных данных, запросов пользователей и работы внутренних алгоритмов. Она разделена на две категории: интерфейсные данные, необходимые для отображения в приложении, представленные в таблице 2.7, и отчетные данные, необходимые для управления системой, которые представлены в таблице 2.8.

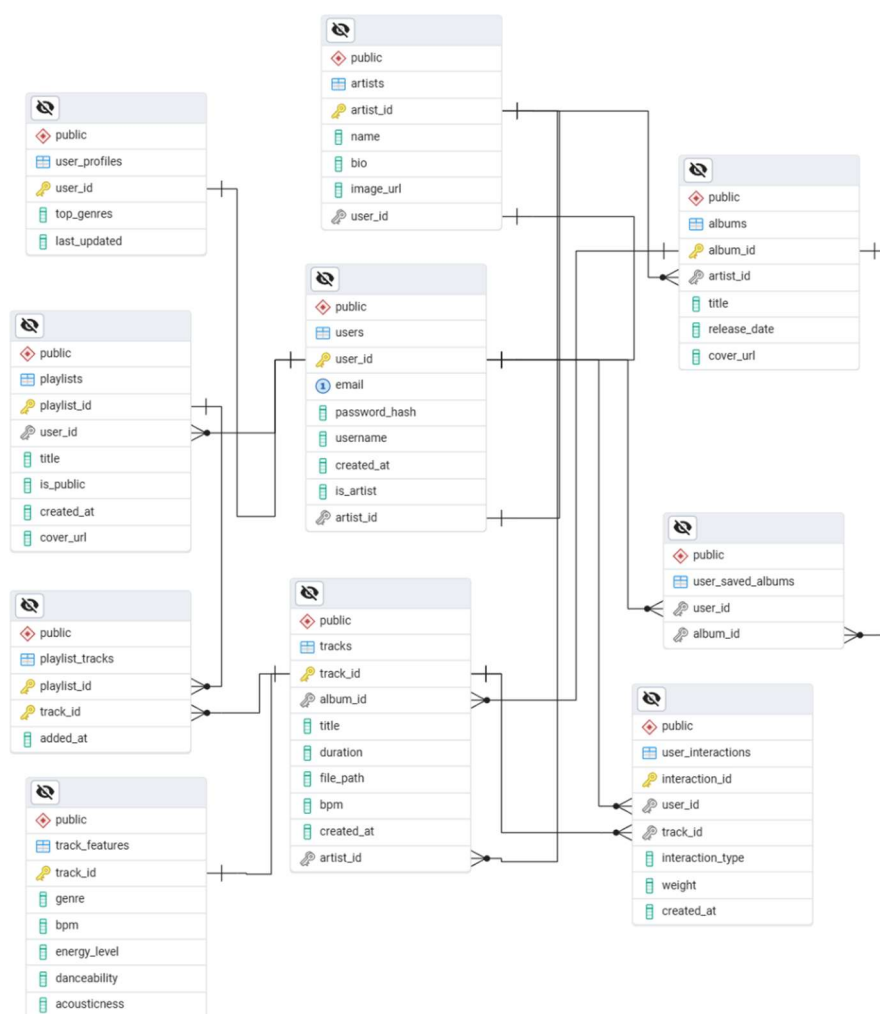


Рисунок 2.4 – Диаграмма базы данных

Таблица 2.3 – Таблицы базы данных для музыкального сервиса

Название	Содержимое	Обновление	Сервисы
<i>users</i>	Учетные данные и профили пользователей	При регистрации или редактировании профиля	Основные
<i>artists</i>	Информация об исполнителях и их описание	При добавлении или изменении данных артиста	Основные
<i>albums</i>	Метаданные альбомов и ссылки на обложки	При загрузке нового альбома	Основные
<i>tracks</i>	Метаданные треков и пути к файлам	При загрузке или обновлении контента	Основные
<i>playlists</i>	Структура и описание плейлистов пользователя	При создании или редактировании плейлиста	Основные
<i>playlist_tracks</i>	Связи треков с плейлистами (ассоциативная)	При добавлении/удалении трека в плейлист	Основные
<i>track_features</i>	Аудио-характеристики для алгоритмов (BPM, энергия)	При загрузке и первичном анализе трека	Дополнительные (Рекомендации)
<i>user_interactions</i>	Логи действий пользователей (прослушивания, лайки)	В режиме реального времени при каждом действии	Дополнительные (Рекомендации)
<i>user_profiles</i>	Агрегированные предпочтения пользователей	Периодически при обновлении интересов	Дополнительные (Рекомендации)

Таблица 2.4 – Структура таблицы «*tracks*» (Основной контент)

Поле	Тип данных	Описание	Источник данных	Периодичность обновления
<i>track_id</i>	<i>INTEGER</i>	Уникальный идентификатор трека	Автогенерация БД	При создании записи
<i>album_id</i>	<i>INTEGER</i>	Идентификатор альбома	Панель управления артиста	При создании записи
<i>title</i>	<i>VARCHAR(255)</i>	Название композиции	Ввод артистом	При загрузке
<i>duration</i>	<i>INTEGER</i>	Длительность трека (сек.)	Метаданные файла	При загрузке
<i>file_path</i>	<i>VARCHAR(500)</i>	Путь к файлу в S3	Сервер обработки контента	При загрузке
<i>bpm</i>	<i>INTEGER</i>	Темп композиции	Анализатор (алгоритм)	При загрузке

Таблица 2.5 – Структура таблицы «*user_interactions*» (Аналитическая логика)

Поле	Тип данных	Описание	Источник данных	Периодичность обновления
<i>interaction_id</i>	<i>INTEGER</i>	Идентификатор записи взаимодействия	Автогенерация БД	В реальном времени
<i>user_id</i>	<i>INTEGER</i>	Идентификатор пользователя	Сессия авторизации	При действии
<i>track_id</i>	<i>INTEGER</i>	Идентификатор трека	Интерфейс плеера	При действии
<i>interaction_type</i>	<i>VARCHAR(20)</i>	Тип действия (<i>like, listen, skip</i>)	Клиентское приложение	В момент события

Продолжение таблицы 2.5

Поле	Тип данных	Описание	Источник данных	Периодичность обновления
<i>weight</i>	<i>INTEGER</i>	Весовой коэффициент действия	Бизнес-логика системы	При событии
<i>created_at</i>	<i>TIMESTAMP</i>	Время события	Системные часы	В момент события

Таблица 2.6 – Описание связей между таблицами базы данных

Название таблицы 1	Название таблицы 2	Описание
<i>artists</i>	<i>albums</i>	Один исполнитель может выпустить несколько альбомов
<i>albums</i>	<i>tracks</i>	В одном альбоме может содержаться несколько треков
<i>users</i>	<i>playlists</i>	Один пользователь может создать несколько плейлистов
<i>playlists</i>	<i>playlist_tracks</i>	В одном плейлисте может быть несколько записей с треками
<i>tracks</i>	<i>playlist_tracks</i>	Один трек может входить в несколько плейлистов (связь многие-ко-многим)
<i>users</i>	<i>user_interactions</i>	Один пользователь может совершить несколько действий (прослушивание, лайк)
<i>tracks</i>	<i>user_interactions</i>	Один трек может быть связан с множеством действий пользователей
<i>tracks</i>	<i>track_features</i>	Каждому треку соответствует одна запись с аналитическими характеристиками
<i>users</i>	<i>user_profiles</i>	Каждому пользователю соответствует один профиль предпочтений

Таблица 2.7 – Выходные данные, предназначенные для пользователей

Тип данных	Структура (формат)	Получатель	Периодичность	Описание
Результаты поиска	Массив <i>JSON</i> -объектов	Пользователь	При каждом запросе	Список треков/альбомов, соответствующих критериям

Продолжение таблицы 2.7

Тип данных	Структура (формат)	Получатель	Периодичность	Описание
Персональные рекомендации	Массив объектов (<i>ID</i> , метаданные)	Пользователь	При обновлении ленты	Список треков, сформированный алгоритмом
Информация о профиле	Объект JSON	Пользователь	При загрузке страницы	Статистика прослушиваний, история
Статус плейлиста	Булево значение, список треков	Пользователь	В режиме реального времени	Подтверждение сохранения/изменения

Таблица 2.8 – Выходные данные для администратора (отчеты)

Наименование	Получатель	Периодичность	Описание
Отчет об активности	Администратор / Лейбл	Ежемесячно	Статистика по количеству прослушиваний конкретных треков
Журнал системных событий	Администратор	При возникновении критических ошибок	Лог-файл (текстовый формат) для отладки
Статистика пользователей	Менеджер проекта	Еженедельно	Данные о регистрации новых пользователей и их предпочтениях

2.5 Техническое и системное программное обеспечение

Для эффективного функционирования проектируемой информационной системы необходимо наличие комплекса технических и программных средств, обеспечивающих стабильную обработку медиаконтента, выполнение алгоритмов рекомендаций и взаимодействие с пользователями.

Техническая база разделена на серверную часть, обеспечивающую работу ядра системы, и клиентскую часть, используемую для доступа к

функционалу. Требования к техническому обеспечению представлены в таблице 2.9.

Таблица 2.9 – Требования к техническому обеспечению

Компонент системы	Характеристика	Требования
Сервер приложений	Процессор	не менее 4-х ядер, частота 2.4 ГГц и выше
	Оперативная память	не менее 16 Гб (для кэширования БД и обработки запросов)
	Дисковая подсистема	SSD (NVMe) объемом от 500 Гб для БД и логов
Сетевое оборудование	Пропускная способность	Канал связи не менее 1 Гбит/с
Клиентское устройство	Процессор	2-х ядерный, частота 1.8 ГГц и выше
	Оперативная память	не менее 4 Гб
	Монитор	Разрешение не менее 1280x720 пикселей

Программный комплекс включает в себя системное и прикладное программное обеспечение из таблицы 2.10. Выбор технологий обусловлен необходимостью обеспечения высокой надежности, масштабируемости и кроссплатформенности.

Таблица 2.10 – Требования к программному обеспечению

Категория	Наименование	Рекомендуемая версия	Обоснование выбора
1	2	3	4
ОС (сервер)	<i>Ubuntu Server</i> 22.04 LTS	64-bit	Стабильность, поддержка <i>Docker</i>
СУБД	<i>PostgreSQL</i>	16.x	Поддержка сложных реляционных запросов и <i>JSONB</i>

Продолжение таблицы 2.10

1	2	3	4
Среда выполнения	<i>JAVA / Node.js</i>	3.12+ / 20.x	Высокая скорость обработки асинхронных запросов
Контейнеризация	<i>Docker</i>	26.x	Обеспечение переносимости окружения
Хранилище данных	<i>MinIO</i> (S3-совместимое)	Последняя стабильная	Эффективное хранение медиа-объектов
Браузер (клиент)	<i>Chrome/Firefox/Edge</i>	Актуальные версии	Поддержка <i>HTML5</i> и <i>Web Audio API</i>

2.6 Эргономическое обеспечение

При проектировании интерфейса музыкальной информационной системы первоочередное внимание уделялось снижению зрительного утомления и повышению скорости считывания информации[7]. В качестве основного стилистического направления выбрана темная цветовая схема, где фоновые пространства рендерятся в глубоких графитовых и антрацитовых тонах. Это техническое решение минимизирует общую яркость экрана и предотвращает усталость глаз при длительной работе в условиях слабой освещенности, что характерно для сценариев студийной работы или вечернего прослушивания аудиоконтента. Для контрастного выделения текстовых блоков и метаданных используется мягкий белый цвет с контролируемой прозрачностью, варьирующейся в зависимости от иерархической важности элемента. Акцентные интерактивные элементы управления, такие как кнопка запуска воспроизведения и целевая кнопка подтверждения загрузки файлов в модуле Студии, окрашены в насыщенный зеленый цвет, что интуитивно фокусирует внимание пользователя на главных функциях текущего экрана[8].

Шрифтовое оформление интерфейса стандартизировано во всех экранных формах для поддержания стилистического единства. В системе применяется современный гротескный шрифт из семейства без засечек (*Sans-Serif*), характеризующийся высокой читаемостью при малых кеглях. Иерархия текстовых сообщений выстроена за счет четкого разграничения размеров и насыщенности начертания. Названия альбомов, крупных разделов и имена

артистов выделяются полужирным начертанием увеличенного размера, тогда как второстепенные метаданные, включая длительность треков, жанровую принадлежность и показатели темпа в *BPM*, отображаются уменьшенным шрифтом в приглушенно-серой гамме. Такое разделение позволяет пользователю бегло сканировать интерфейс медиатеки и мгновенно вычленять нужную информацию без совершения лишних поисковых действий.

Компоновка элементов управления спроектирована по модульному принципу на основе устоявшихся паттернов поведения пользователей. Экранное пространство разделено на три перманентные зоны, взаимное расположение которых остается неизменным в процессе навигации. Левая часть отведена под статичное вертикальное меню навигации и списки личной медиатеки слушателя. Это обеспечивает постоянный и быстрый доступ к пользовательским плейлистам и сохраненным альбомам из любой точки приложения. Центральная область функционирует как динамически обновляемый контентный блок, геометрия которого подстраивается под тип выводимых данных, табличная сетка композиций или структурированная форма ввода в модуле администратора. Нижняя зона экрана жестко зарезервирована под медиаплеер, консолидирующий элементы управления воспроизведением, ползунок хронометража и модуль регулировки громкости.

Выбор конкретных элементов управления основывался на минимизации количества кликов, необходимых для выполнения целевого действия. Вместо перегруженных текстовых меню в системе активно применяются интуитивно понятные графические пиктограммы. Контекстные действия, такие как добавление аудиозаписи в плейлист, скрыты за компактными кнопками вызова всплывающих меню, которые появляются непосредственно в строке выбранного трека. Для ввода структурированной информации в модуле Студии применены четко очерченные текстовые поля со встроенными подсказками, которые исчезают при начале ввода, и выпадающие списки для исключения ручных ошибок при связывании сущностей. Все интерактивные компоненты обладают увеличенной областью клика (*padding*), что исключает случайные промахи курсора и повышает общую эргономику взаимодействия с системой.

Удобство работы с разработанной информационной системой поддерживается развитой подсистемой обратной связи и валидации. Программная среда непрерывно информирует пользователя о текущем состоянии процессов с помощью неблокирующих всплывающих уведомлений и статусных сообщений. При совершении критических или длительных операций, таких как *multipart*-загрузка тяжелого *.mp3* файла в объектное хранилище *MinIO S3*, интерфейс блокирует повторную отправку формы и отображает анимированный индикатор прогресса, предотвращая ошибочные

действия пользователя. В случае возникновения исключений, вызванных сетевыми сбоями или недоступностью эндпоинтов *API* бэкенда, система не прекращает аварийно свою работу, а выводит локальное информативное сообщение с предложением повторить операцию, аккуратно обрабатывая сетевые ответы в фоновом режиме.

Экспертная оценка разработанного графического интерфейса подтверждает его полное соответствие современным требованиям эргономики программных средств. В системе строго выдержан единый визуальный стиль оформления: геометрия окон, скругления углов элементов, поведение кнопок при наведении и цветовые маркеры идентичны во всех модулях, включая пользовательский интерфейс прослушивания и расширенный интерфейс Студии. Постоянное присутствие главных навигационных узлов на экране исключает дезориентацию пользователя внутри структуры приложения. Система успешно реализует концепцию адаптивного веб-интерфейса, сохраняя пропорции, читаемость шрифтов и доступность элементов управления при изменении разрешения экрана монитора, что гарантирует безопасную, комфортную и высокопроизводительную работу с разработанным программным продуктом.

2.7 Организационное обеспечение

Организационное обеспечение проектируемой ИС определяет правила взаимодействия пользователей с системой, уровни доступа к функциональным возможностям и комплекс мер по защите данных. Целью данного обеспечения является поддержание работоспособности системы и предотвращение несанкционированного доступа к медиаконтенту и пользовательской информации.

Для обеспечения гибкого управления системой реализована модель контроля доступа на основе ролей (*RBAC — Role-Based Access Control*). Это позволяет разграничить ответственность и гарантировать, что каждый пользователь работает только с доступными ему функциями из таблицы 2.11.

Организационные меры безопасности направлены на защиту интеллектуальной собственности артистов и конфиденциальности пользовательских данных. Меры представлены в таблице 2.12.

Доступ к системе организован через унифицированный *API*-шлюз, который определяет права сессии в зависимости от учетной записи:

- 1 Публичный доступ. Осуществляется через веб-сайт или мобильное приложение без авторизации. Доступны только страницы каталога и поиск (с ограничениями на воспроизведение).

2 Пользовательский доступ. Осуществляется через авторизованную сессию (*JWT*-токены). Дает право на доступ к личному кабинету, сохранение плейлистов и полнофункциональный стриминг.

3 Административный доступ. Осуществляется через защищенную административную панель с обязательным использованием двухфакторной аутентификации (желательно) или защищенного канала доступа (*VPN/IP-whitelist*).

Таблица 2.11 – Классификация пользователей и права доступа

Роль пользователя	Описание	Основные права
Пользователь (Слушатель)	Зарегистрированный клиент платформы	Просмотр контента, поиск, создание плейлистов, прослушивание треков
Артист (Контент-мейкер)	Автор музыкальных произведений	Загрузка треков, управление метаданными альбомов, просмотр статистики
Администратор	Оператор системы	Управление пользователями, модерация контента, настройка системных параметров, просмотр логов

Таблица 2.12 – Комплекс мер по защите информации

Направление защиты	Мера безопасности	Реализация
1	2	3
Конфиденциальность	Шифрование передаваемых данных	Использование протокола <i>HTTPS (TLS 1.3)</i> для всех запросов
Целостность	Защита от <i>SQL</i> -инъекций	Использование <i>ORM</i> и параметризованных запросов в коде
Аутентификация	Безопасное хранение паролей	Хеширование паролей с использованием алгоритма <i>bcrypt</i> (солирование)

Продолжение таблицы 2.12

1	2	3
Доступ к контенту	Ограничение доступа к файлам	Генерация пре-сайдн (временных) ссылок на S3-объекты
Сохранность	Резервное копирование	Ежедневное создание снимков базы данных и версионирование контента
Мониторинг	Аудит действий	Логирование всех критических операций (вход, изменение прав, удаление контента)

3 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ ИНФОРМАЦИОННОЙ СИСТЕМЫ

3.1 Выбор программных средств реализации ИС

Для обеспечения высокой производительности, масштабируемости и модульности разрабатываемой информационной системы (музыкальной стриминговой платформы) выбрана современная многокомпонентная архитектура. Ниже приведено описание и обоснование выбора программных средств для каждого уровня системы.

В качестве основного технологического стека для разработки серверной логики выбран *Java* и фреймворк *Spring Boot*.

Язык программирования *Java* (версия 17/21). Обеспечивает строгую типизацию, платформонезависимость и высокую надежность при работе с многопоточностью (что критично для стриминга аудиосигналов).

Фреймворк *Spring Boot*. Использование экосистемы *Spring* (*Spring Data JPA*, *Spring Web*) позволило ускорить процесс разработки за счет автоматической конфигурации компонентов, реализовать эффективную архитектуру *REST API* для взаимодействия с клиентской частью и обеспечить изоляцию транзакций при работе с базой данных с помощью декларативного управления (*@Transactional*)[9].

В качестве альтернативы рассматривался *Node.js* (*JavaScript*). Однако для задач, связанных с анализом аудиопотоков (подсчет *BPM*, извлечение аудио-фич), обработкой транзакций и построением отказоустойчивой архитектуры, связка *Java/Spring Boot* является более стабильным и производительным решением.

Для реализации пользовательского интерфейса выбраны стандартные веб-технологии: *HTML5*, *CSS3* и *JavaScript* (*Vanilla JS*) без использования тяжелых фреймворков (класса *React/Angular*).

HTML5 и *CSS3* позволили создать адаптивный, отзывчивый интерфейс в темных тонах (индустриальный стандарт медиаплееров) с использованием современных механизмов верстки (*Flexbox*, *Grid Layout*).

Асинхронное взаимодействие с сервером реализовано через встроенный интерфейс *JavaScript Fetch API*.

Для хранения структурированных данных (информация о пользователях, треках, альбомах и аудио-характеристиках) выбрана реляционная СУБД *PostgreSQL*, потому что она обеспечивает полную поддержку принципов *ACID*, гарантируя сохранность данных, обладает высокой производительностью при выполнении сложных выборки и операций

связывания таблиц (*JOIN*), что критично для сквозного живого поиска по системе и отлично интегрируется со *Spring Data JPA / Hibernate*.

Для хранения тяжелых аудиофайлов (*MP3/WAV*) выбрано объектное хранилище с *S3*-совместимым *API MinIO*.

Объектные хранилища организованы эффективнее традиционных файловых систем ОС при работе с миллионами медиафайлов.

Также позволяет организовать стриминг аудио на клиентскую часть по частям (чанками), снижая нагрузку на оперативную память сервера.

Хранить аудиофайлы напрямую в реляционной БД в виде *BLOB* неэффективно, так как это раздувает размер базы и тормозит выборки. Хранение в локальной папке сервера ограничивает горизонтальное масштабирование. *S3*-совместимое хранилище решает обе эти проблемы.

Выбранный стек технологий представляет собой сбалансированное решение, удовлетворяющее требованиям безопасности, отказоустойчивости и высокой скорости отклика веб-интерфейса при глобальном поиске и воспроизведении медиаконтента.

3.2 Структура программного обеспечения ИС

Программное обеспечение разрабатываемой информационной системы имеет модульную трехзвенную архитектуру и четко разделено на клиентское приложение (*Frontend*) и серверную часть (*Backend*). Взаимодействие между ними осуществляется по протоколу *HTTP* с использованием архитектурного стиля *REST* [10].

На рисунке 3.1 представлена структурная схема программного обеспечения ИС, отражающая основные компоненты, внутренние модули и их взаимосвязи.

Представленная структурная схема иллюстрирует статическую декомпозицию программного обеспечения музыкального стримингового сервиса на три основополагающие категории, определяющие архитектурную целостность продукта: логические модули, системные библиотеки и файлы ресурсов. В отличие от простого представления структуры исходного кода на уровне каталогов, данная схема отражает компонентное разделение системы по функциональному признаку, где каждый элемент решает строго определенную прикладную задачу, обеспечивая независимость и слабую связанность частей ИС.

Центральное место в архитектуре занимают программные модули, инкапсулирующие ключевую бизнес-логику стриминговой платформы.

Модуль глобального поиска и обработки медиаданных является связующим звеном между пользовательским интерфейсом и базой данных при выполнении поисковых запросов. Внутри него выделены две важнейшие подсистемы. Первая подсистема реализует параллельный асинхронный опрос программных интерфейсов (*API*) треков и альбомов. Использование асинхронного подхода позволяет одновременно отправлять запросы к разным таблицам БД, что существенно сокращает время ожидания ответа для пользователя. Вторая подсистема отвечает за динамическую генерацию элементов объектной модели документа (*DOM*). Полученный от сервера структурированный ответ в формате *JSON* на лету преобразуется в графические компоненты поисковой выдачи, разделяя экран на приоритетную зону треков и нижнюю область альбомов без перезагрузки всей страницы.

Модуль цифрового анализа аудио функционирует на стыке системного программирования и обработки сигналов. Процедуры извлечения метаданных и парсинга аудиосигналов считывают бинарный поток загружаемых файлов, определяя технические параметры аудиозаписи. На основе этих данных срабатывает процедура автоматического подсчета темпа трека, которая математически анализирует частотные пики и вычисляет точное значение BPM (ударов в минуту). Данная метрика впоследствии используется системой для формирования рекомендаций и фильтрации контента.

Модуль потокового вещания организует непрерывную передачу медиаконтента с сервера на клиентскую сторону. Он обрабатывает сетевые запросы к аудиофайлам, разбивая их на небольшие дискретные части (чанки). Побайтовая отдача аудиосигнала предотвращает переполнение оперативной памяти сервера, так как файл не загружается в нее целиком, а транслируется в виде потока, который плеер на стороне клиента начинает воспроизводить незамедлительно.

Модуль объектного хранилища изолирует низкоуровневые операции с файловой системой и облачной инфраструктурой. Он предоставляет абстрактный интерфейс для взаимодействия с бакетами *S3*, отвечая за безопасную загрузку, хранение и проверку целостности тяжеловесных медиафайлов.

Модуль доступа к данным формирует объектно-реляционное отображение (*DAO*). Этот компонент транслирует высокоуровневые запросы приложения в оптимизированные *SQL*-команды. Важной особенностью подсистемы является применение специализированных графов сущностей, которые подавляют избыточные запросы к БД и принудительно инициализируют связанные объекты в обход ленивых прокси-объектов, исключая появление пустых связей при передаче данных на фронтенд.

Второй вертикальный столбец схемы объединяет внешнее программное обеспечение и фреймворки, на фундаменте которых развернута информационная система.

Spring Boot Core выступает в качестве главного каркаса серверной части ИС, обеспечивая управление жизненным циклом компонентов, автоматическое внедрение зависимостей и диспетчеризацию входящих *HTTP*-запросов.

Jackson Databind решает задачу сериализации данных, преобразуя сложные структуры объектов *Java* в текстовый формат *JSON* для обмена информацией с фронтендом. Программная настройка этой библиотеки позволяет игнорировать пустые прокси-объекты базы данных, предотвращая зацикливание при сборке ответа.

MinIO Java SDK предоставляет набор готовых процедур и протоколов для прямой интеграции серверного кода с *S3*-совместимым хранилищем, избавляя от необходимости вручную формировать низкоуровневые сетевые пакеты.

Project Lombok оптимизирует исходный код приложения, автоматически генерируя рутинные методы (геттеры, сеттеры, конструкторы классов) на этапе компиляции, что разгружает программную архитектуру и повышает читаемость кода.

Правая часть схемы консолидирует декларативные и конфигурационные файлы, необходимые для корректного развертывания и внешнего отображения ИС.

Статические файлы интерфейса включают в себя разметку и каскадные таблицы стилей, которые задают визуальный облик веб-приложения, формируя темную тему плеера и адаптивное расположение элементов.

Графические ресурсы содержат локальные векторные изображения, интегрированные непосредственно в программный код клиентской части. Они служат защитным механизмом: если внешние сервера хранения картинок недоступны или возвращают сетевые ошибки, модуль динамического рендеринга подставляет эти встроенные векторные заглушки, сохраняя стабильность и визуальную целостность интерфейса.

Конфигурации среды представлены внешними конфигурационными файлами, где в виде пар «ключ-значение» зафиксированы глобальные параметры функционирования ИС, такие как адреса сетевых портов, доступы к СУБД *PostgreSQL* и секретные ключи авторизации в объектном хранилище. Такое вынесение настроек позволяет изменять параметры работы системы без необходимости перекомпиляции программного кода.

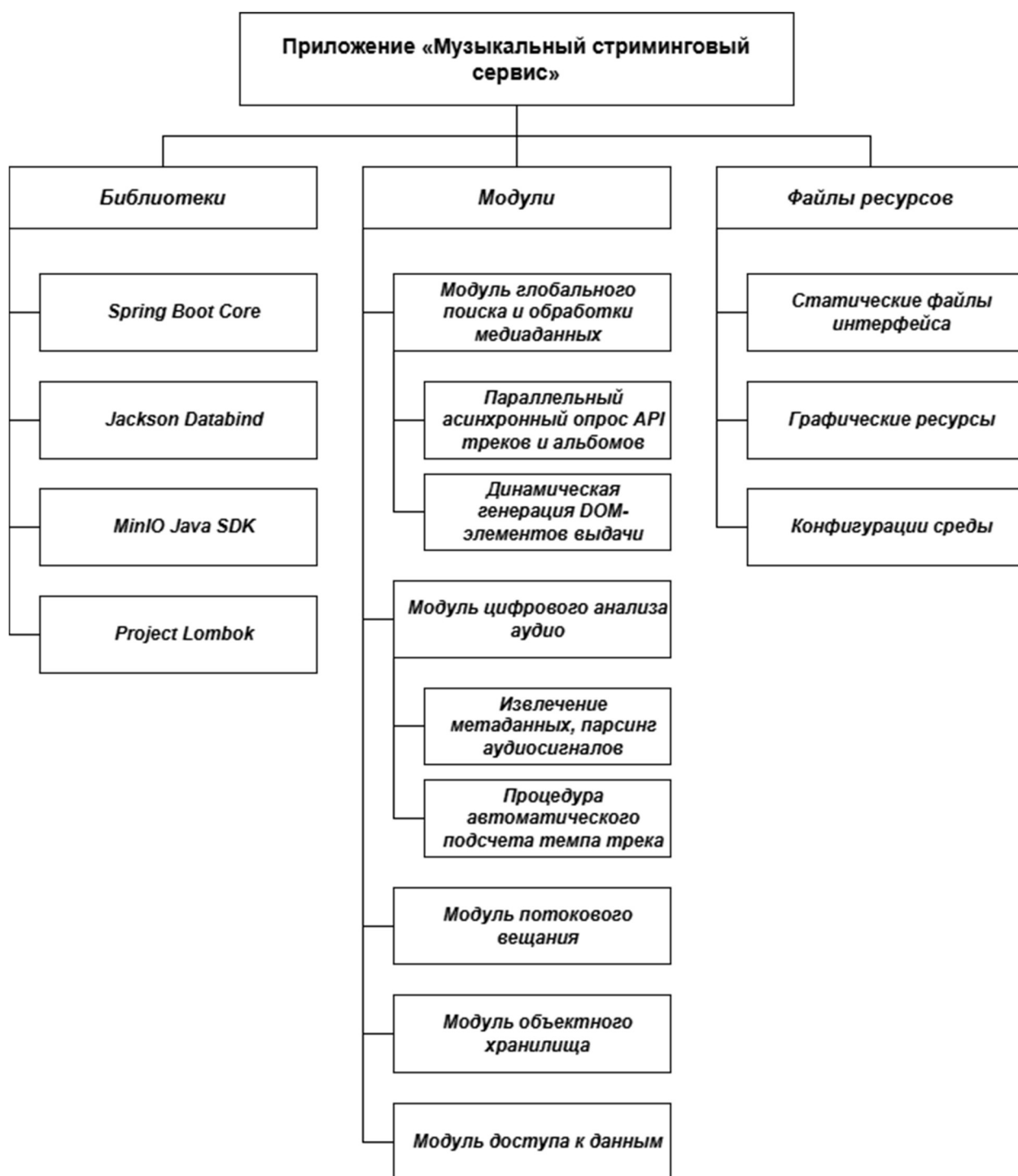


Рисунок 3.1 – Схема программного обеспечения ИС

3.3 Разработка программного кода ИС

Процесс разработки программного кода информационной системы базировался на принципах объектно-ориентированного системного проектирования и итерационного воплощения архитектурных моделей. На этапе проектирования сформирована высокоуровневая спецификация требований и поведенческих сценариев, которая впоследствии транслирована

в статическую структуру классов и динамические модели межкомпонентного взаимодействия. Визуализация проектных решений выполнена с использованием унифицированного языка моделирования *UML*.

Начальный этап системного проектирования сосредоточен на определении границ информационной системы и формализации ролевой модели взаимодействия. Поведенческая структура сервиса зафиксирована на диаграмме вариантов использования (*Use Case Diagram*), представленная на рисунке 3.2, которая отражает взаимосвязи между внешними актерами и ключевыми сервисами платформы.

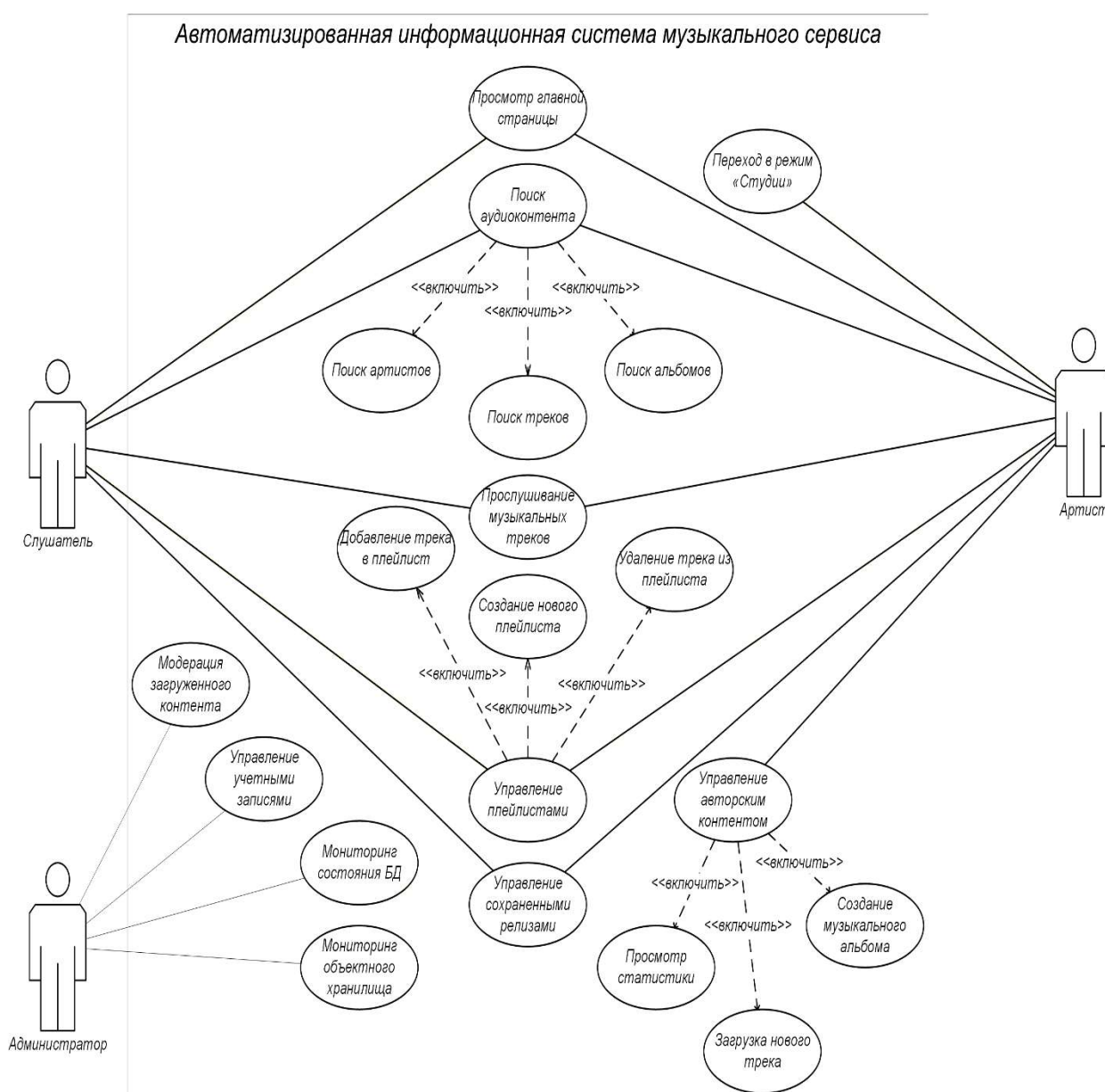


Рисунок 3.2 – Диаграмма вариантов использования

Главным актером системы выступает Пользователь, для которого спроектирован набор базовых прецедентов, таких как «Аутентификация в системе», «Управление личной медиатекой», «Прослушивание аудиопотока» и «Выполнение глобального поиска». Прецедент выполнения поиска находится в отношении расширения (*extend*) со специфичными сценариями вывода информации, разделяющими поисковую выдачу на зоны треков и альбомов. В свою очередь, прецедент загрузки аудиофайлов, доступный администратору, находится в отношении включения (*include*) с автоматическим запуском подсистемы цифрового анализа сигналов, поскольку сохранение трека в системе невозможно без предварительного извлечения его физических характеристик и расчета темпа.

Переход от функциональных требований к программной реализации осуществлен посредством построения диаграммы классов (*Class Diagram*), представленной на рисунке 3.3, которая служит основой для генерации таблиц реляционной базы данных средствами *ORM Hibernate* и определяет логику объектно-реляционного отображения.

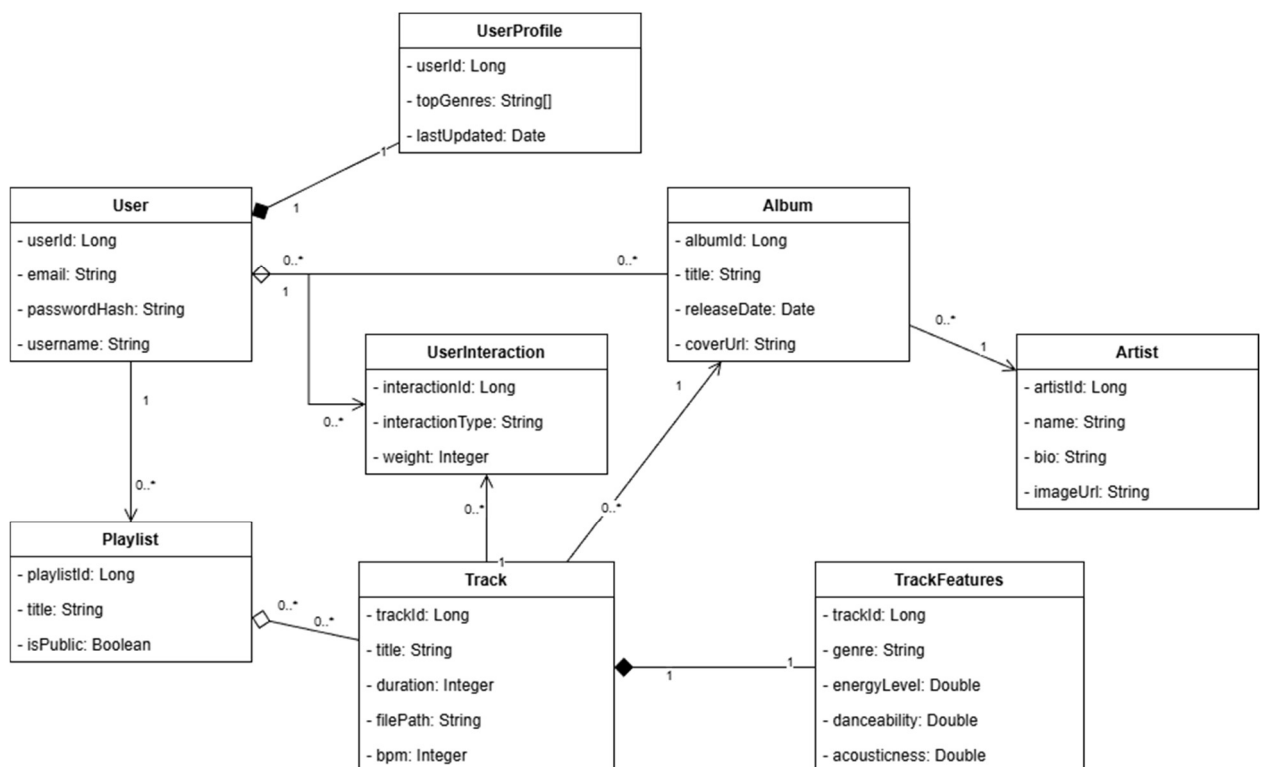


Рисунок 3.3 – Диаграмма классов

Центральным элементом объектной модели является класс *Track*, содержащий метаданные аудиозаписи, такие как наименование, длительность и уникальный ключ ссылки на бинарный объект в хранилище *S3*. Класс *Track*

находится в отношении ассоциации типа «многие к одному» (*Many-to-One*) с классом *Album*, который аккумулирует информацию об издании, обложке и связанном авторе. Для оптимизации структуры и исключения избыточности, специфичные вычисляемые параметры звуковой волны вынесены в изолированный класс *TrackFeatures*, содержащий поле темпа (*BPM*) и связанный с базовым треком отношением «один к одному» (*One-to-One*). Взаимодействие пользователя с контентом фиксируется через класс *User*, который агрегирует коллекции выбранных треков и альбомов, формируя реляционные таблицы связей для долгосрочного хранения персистентного состояния медиатеки.

Процесс выполнения сквозного асинхронного поиска и последующего воспроизведения медиаконтента детально описан на диаграмме последовательности (*Sequence Diagram*), представленной на рисунке 3.4, отражающей временную шкалу вызова процедур и передачи сообщений между фронтендом, бэкендом и внешними хранилищами.

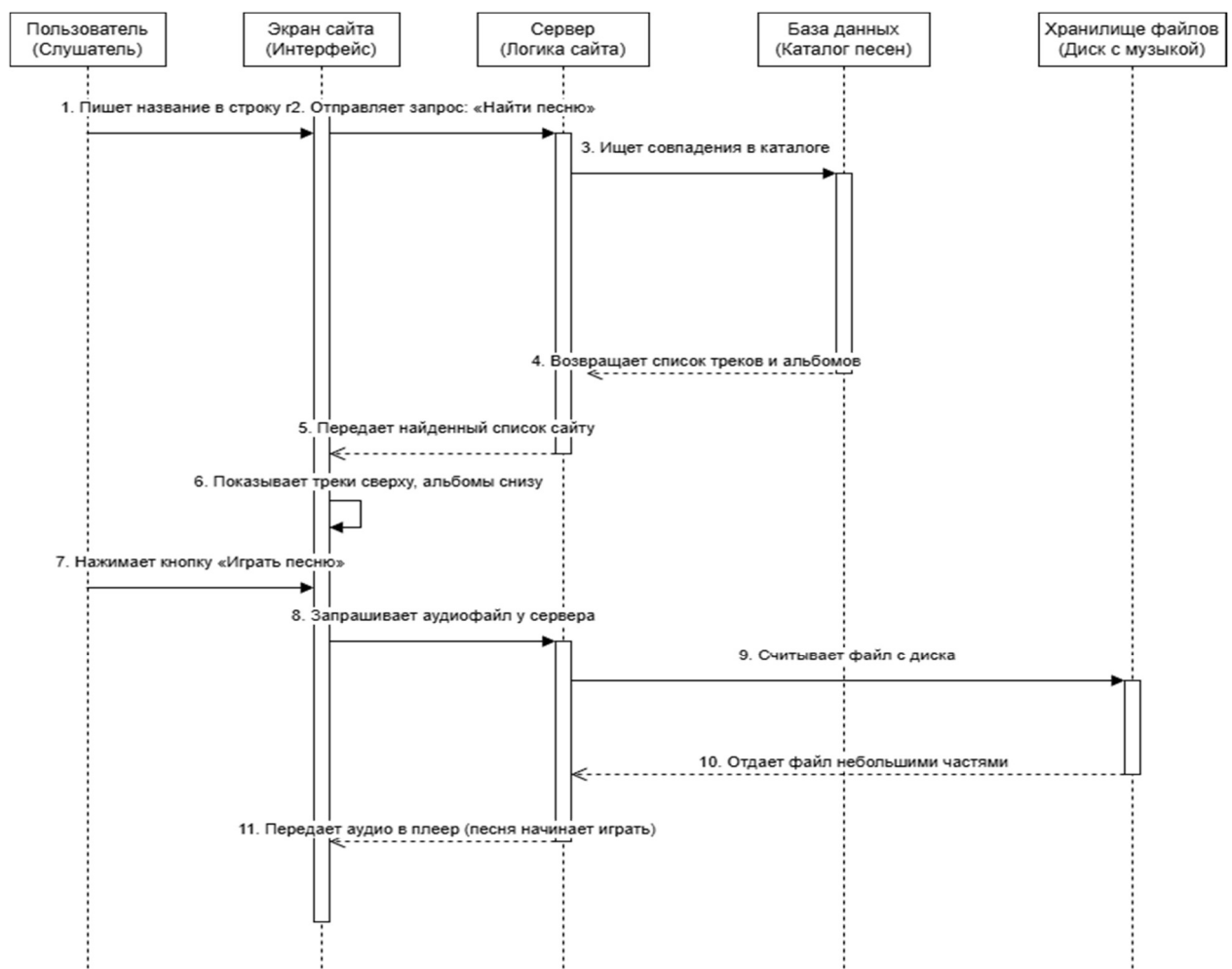


Рисунок 3.4 – Диаграмма последовательности

Динамический сценарий активизируется в момент ввода пользователем текстового запроса в поисковую строку интерфейса. Асинхронный сетевой клиент фронтенда инициирует вызов процедуры *handleSearch*, которая генерирует параллельные *HTTP*-запросы к программным эндпоинтам *TrackController* и *AlbumController*. На стороне бэкенда управление передается в сервисный слой, где метод *TrackService.search()* обращается к интерфейсу репозитория. Слой доступа к данным выполняет оптимизированный *SQL*-запрос к СУБД *PostgreSQL*, задействуя настроенные графы сущностей для одновременного извлечения связанных объектов альбомов и рассчитанных фич *BPM* в обход отложенной инициализации *Hibernate*-прокси.

Полученный структурированный ответ преобразуется библиотекой *Jackson* в пакет данных *JSON* и возвращается на клиентскую сторону, где модуль манипуляции *DOM* динамически перестраивает интерфейс выдачи. При последующем клике пользователя на элемент трека, встроенный браузерный объект *Audio* отправляет потоковый запрос на эндпоинт вещания, который через *StorageService* считывает файл из объектного хранилища *MinIO S3* и транслирует его обратно на клиент в виде непрерывного байтового потока (чанков).

Одним из важнейших алгоритмов в системе является метод для реализации транслирования треков с облачного сервиса.

Для обеспечения непрерывного воспроизведения аудиофайлов и поддержки возможности перемотки трека на стороне клиента (в веб-плеере) разработан контроллер потокового вещания. Метод *streamTrack* обрабатывает как стандартные запросы на полную загрузку файла, так и запросы на частичное чтение контента (*HTTP Range Requests*).

Код метода *streamTrack* представлен в приложении А и разобран ниже:

Когда пользователь нажимает кнопку «Играть», сервер первым делом идет в облачное хранилище и находит там нужную песню по её *ID*.

Он открывает к файлу поток (*inputStream*), по которому пойдут байты файла. Затем измеряет полный размер песни (*fileLength*), чтобы понимать объемы работы. Смотрит на запрос от браузера (*ranges*) – нет ли там требования включить песню не с начала, а с конкретной секунды.

Если в запросе от браузера пусто, значит, песню включают с первой секунды. Включается классический режим.

Сервер создает буфер объемом 4 килобайта и записывает в него часть файла. После это отправляет его пользователю и цикл повторяется. Таким образом серверу не нужно держать всю песню в своей памяти, он работает как транзитный конвейер.

Если пользователь кликнул мышкой на середину трека, срабатывает следующий код.

Сервер высчитывает точную точку старта (*start*) и финиша (*end*), а также сколько всего байт надо отрезать (*rangeLength*).

Команда *InputStream.skip(start)* пропускает в потоке необходимое число байт.

Следующий фрагмент отвечает за прецизионную вычитку строго заданного диапазона байт и трансляцию их в выходной сокет без забивания буфера:

Объект передается *Spring*-контейнеру, который выполняет тело лямбды в отдельном пуле потоков *TaskExecutor*, освобождая поток *Tomcat*(сервер). Запись идет напрямую в поток вывода.

Переменная *bytesToRead* инициализируется размером запрашиваемого диапазона (*rangeLength*). Чтобы при чтении последнего чанка не захватить лишние байты, идущие за пределами верхней границы *end*, размер буфера динамически урезается: *Math.min(buffer.length, bytesToRead)*

Статус 206 *Partial Content* сигнализирует аудио-плееру браузера, что тело ответа – это не весь файл, а лишь фрагмент.

Стриминг – процесс нестабильный. Сетевой сбой между сервером и *MinIO S3*, досрочный разрыв *TCP*-соединения со стороны пользователя (ушел со страницы) или передача некорректного (битого) *ID* трека генерируют исключения (*IOException*, *AmazonS3Exception* и др.). Блок *catch (Exception e)* агрегирует их, не позволяя исключению проброситься на уровень контейнера сервлетов.

3.4 Руководство пользователя

Взаимодействие пользователя с информационной системой (ИС) начинается с этапа авторизации. При запуске клиентской части приложения система инициализирует сессию и запрашивает учетные данные, как показано на рисунке 3.5. В случае успешной аутентификации пользователю открывается доступ к главному интерфейсу медиатеки, работающему в рамках двух основных режимов: стандартного (пользовательского) и расширенного (режима Студии).

Стандартный режим предназначен для навигации, управления персональной медиатекой и воспроизведения аудиоконтента. Переключение в режим Студии, представленный на рисунке 3.6, осуществляется с помощью управляющего элемента «В Студию» в верхней панели интерфейса. Данный

режим активирует административные функции и открывает доступ к модулю загрузки треков и управления альбомами.

Основная прикладная задача пользователя заключается в формировании единого музыкального пространства. Для добавления трека в персональный плейлист пользователь нажимает кнопку контекстного меню рядом с выбранной аудиозаписью, после чего система выполняет запрос к *API*, связывая уникальные идентификаторы плейлиста и трека в базе данных. Обновление содержимого боковой панели медиатеки происходит динамически без перезагрузки страницы благодаря обработке подсистемы реактивного отображения данных.

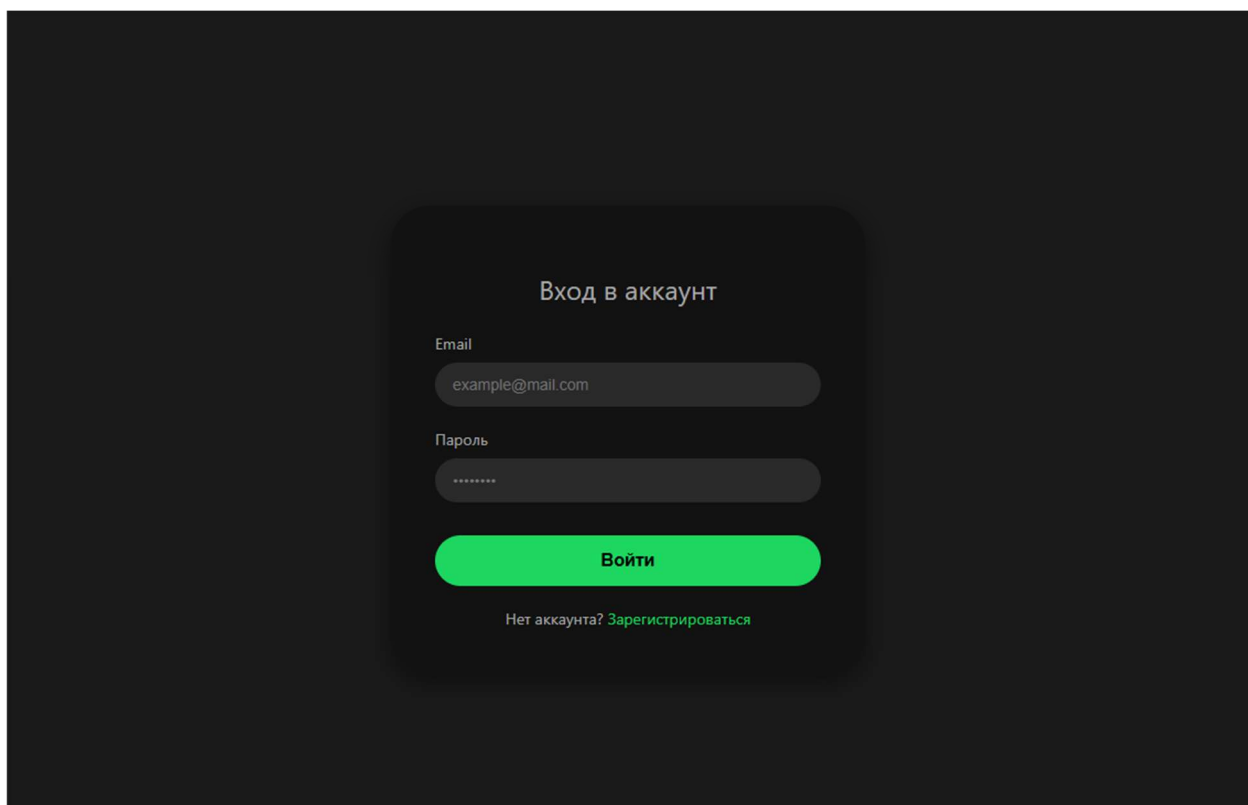


Рисунок 3.5 – Страница авторизации

В процессе эксплуатации ИС могут возникать критические ситуации, связанные с аппаратными сбоями или сетевыми ограничениями. При возникновении ошибок программная среда обрабатывает их на уровне интерфейса, исключая аварийное завершение работы приложения.

Если при отправке запроса (например, при попытке добавить трек в плейлист или открыть профиль артиста) интерфейс сигнализирует об отсутствии ресурса или неверном методе вызова, пользователю необходимо проверить стабильность сетевого соединения и обновить страницу для синхронизации роутинга. Если ошибка повторяется, как показано на

рисунке 3.7 это свидетельствует о проведении технических работ на стороне бэкенд-сервиса.

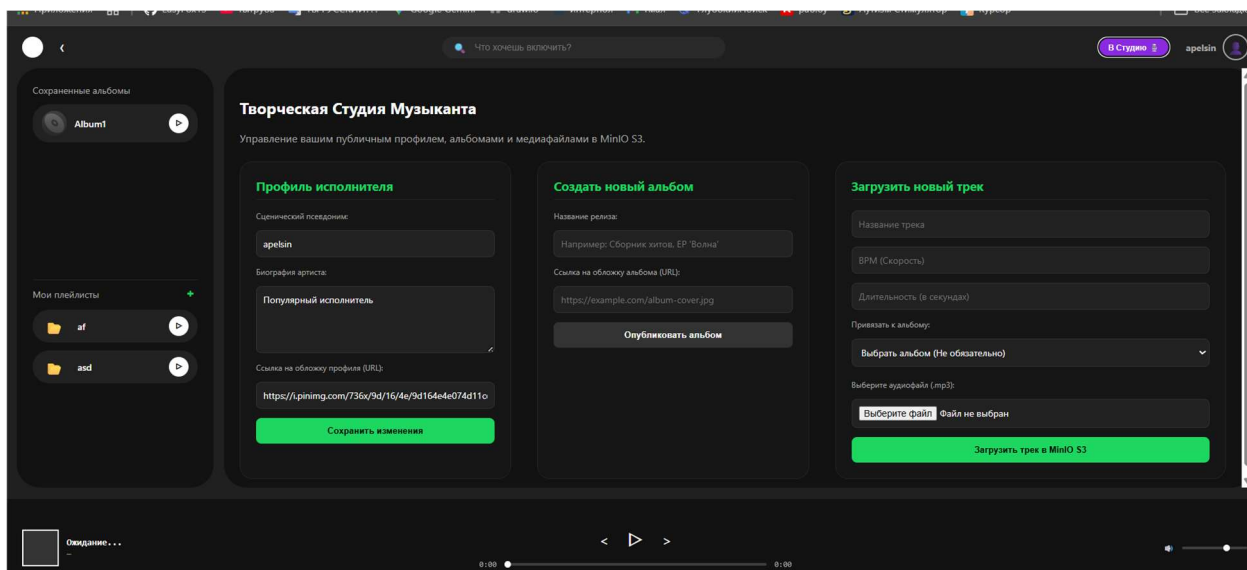


Рисунок 3.6 – Режим студии

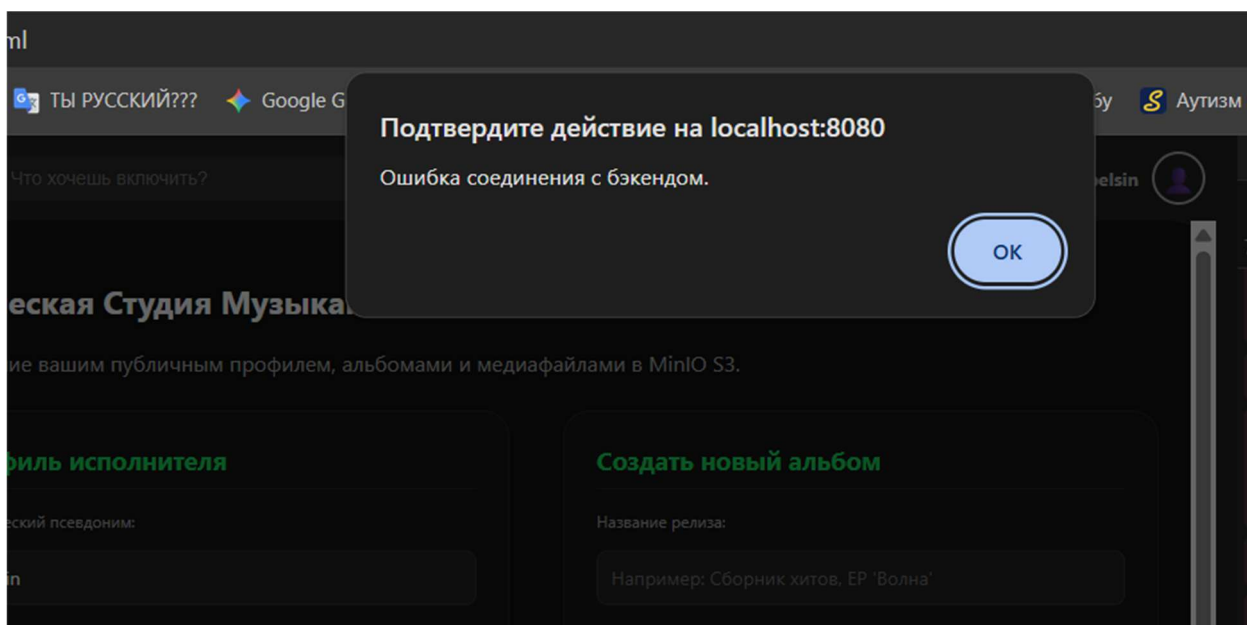


Рисунок 3.7 – Ошибка соединения

В случае прерывания передачи данных от сервера, приводящего к невозможности прочитать *JSON*-ответ (например, при получении оборванной строки данных), на экране отобразится индикатор бесконечной загрузки. Для восстановления корректной работы пользователю следует очистить кэш браузера и повторить запрос.

Если при передаче аудиофайла в хранилище произошло сетевое разъединение, система заблокирует отправку поврежденной формы. Пользователю необходимо убедиться, что исходный файл соответствует формату *.mp3*, не поврежден, и повторно инициировать процедуру отправки через форму Студии.

3.5 Описание интерфейса

Графический интерфейс информационной системы реализован в виде адаптивного веб-приложения с использованием темной цветовой палитры, обеспечивающей снижение зрительной нагрузки при длительной работе. Архитектура интерфейса разделена на три функциональные зоны: навигационную панель, основную рабочую область и модуль административного управления (Студию).

При входе в систему инициализируется основное окно приложения. В левой части экрана располагается статичная навигационная панель (сайдбар), содержащая интерактивные элементы для быстрого доступа к разделам «Главная», «Поиск» и «Ваша медиатека». Под ними отображается динамический список пользовательских плейлистов и сохраненных альбомов.

Центральная область предназначена для отображения контента. При переходе в плейлист или альбом в верхней части рабочей зоны выводятся графический баннер (обложка), текстовые поля с названием, автором и датой релиза. Ниже располагается табличная сетка со списком треков, где каждый элемент снабжен кнопкой воспроизведения, текстовыми полями с названием композиции, длительностью и кнопкой вызова контекстного меню для добавления трека в персональные плейлисты.

4 ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ И РЕАЛИЗАЦИИ НА РЫНКЕ АВТОМАТИЗИРОВАННОЙ ИНФОРМАЦИОННОЙ СИСТЕМЫ МУЗЫКАЛЬНОГО СЕРВИСА

4.1 Характеристика музыкального сервиса, разрабатываемого для реализации на рынке

Разрабатываемая информационная система представляет собой специализированное программное решение для автоматизации управления медиабibliothекми и анализа музыкального контента. Основной целью проекта является создание единой цифровой среды для хранения, индексации и интеллектуальной обработки аудиофайлов, что позволяет независимым музыкальным лейблам и медиа-платформам эффективно управлять авторским правом и пользовательским опытом. В рамках своих функций система обеспечивает централизованное хранение данных в S3-совместимых облачных структурах и проводит автоматический расчет технических параметров треков, таких как *BRM* и тональность.

Профиль целевой аудитории охватывает владельцев независимых музыкальных каталогов и агрегаторов медиаконтента. Актуальная потребность в продукте подтверждается дефицитом доступных инструментов автоматизации для малых правообладателей, вынужденных конкурировать с такими глобальными аналогами, как *Spotify*, *Yandex Music* и специализированными системами управления медиа-активами.

На основании анализа рыночного спроса прогнозируемый объем продаж в течение первого расчетного года составляет шестьдесят корпоративных лицензий и пятьсот индивидуальных годовых подписок. Отпускная цена одной корпоративной лицензии определена на уровне ста пятидесяти тысяч белорусских рублей, что соответствует рыночным ценам на аналогичные системы управления цифровым контентом. В качестве основной модели монетизации выбрано сочетание продажи лицензий для B2B-сектора и Freemium-модели с платной расширенной подпиской для индивидуальных пользователей, что позволяет гибко масштабировать доход. Основными каналами продаж программного продукта станут прямые контракты с организациями и размещение в цифровых магазинах приложений для охвата массового потребителя. Стратегия продвижения продукта базируется на цифровом маркетинге и демонстрации преимуществ в части снижения издержек на инфраструктуру за счет использования открытых облачных стандартов.

Проектирование и внедрение любой ИС связано с затратами материальных, трудовых и финансовых ресурсов, что требует предварительной оценки целесообразности проведения работ[12].

Прямой экономический эффект заключается в снижении совокупной стоимости владения (ТСО) информационной системой за счет оптимизации алгоритмов обработки медиаданных и использования свободно распространяемого ПО (Open Source), что снижает затраты на аренду облачной инфраструктуры.

Экономический эффект для организации-разработчика заключается в получении прибыли с реализации каждой лицензии продукта, а также в снижении совокупной стоимости владения системой за счет архитектурной оптимизации стриминга контента, минимизирующей расходы на эксплуатацию облачной инфраструктуры.

4.2 Расчет инвестиций в разработку музыкального сервиса для его реализации на рынке

Инвестициями для организации-разработчика программного средства являются затраты на его разработку, которые рассчитываются в соответствии с методикой, представленной в таблице 1.

Таблица 1 – Расчет затрат на разработку музыкального сервиса

Наименование статьи затрат	Формула/таблица для расчета	Значение, р.
1. Основная заработная плата разработчиков (Z_o)	Формула (1), таблица 2	31 801,50
2. Дополнительная заработная плата разработчиков (Z_d)	Формула (2)	3 180,15
3. Отчисления на социальные нужды ($P_{соц}$)	Формула (3)	12 103,65
4. Прочие расходы ($P_{пр}$)	Формула (4)	9 540,45
5. Расходы на реализацию	Формула (5)	1 590,08
6. Общая сумма затрат на разработку и реализацию	Формула (6)	58 215,83

Для определения общей суммы основной заработной платы Z_o используется произведение коэффициента премий и стимулирующих выплат $K_{пр}$ на суммарные трудозатраты всех категорий исполнителей, выраженные в денежном эквиваленте, используя формулу 1. В данном расчете значение коэффициента принято равным 1,5, что соответствует надбавке в размере 50%

к базовому окладу. Таким образом полный расчет затрат на основную заработную плату команды разработчиков представлен на таблице 2.

$$Z_o = K_{\text{пр}} \sum_{i=1}^n Z_{\text{чи}} \cdot t_i, \quad (1)$$

где $K_{\text{пр}}$ –коэффициент премий и иных стимулирующих выплат;
 n –категории исполнителей, занятых разработкой программного средства;
 $Z_{\text{чи}}$ –часовой оклад плата исполнителя i -й категории, р.;
 t_i –трудоемкость работ, выполняемых исполнителем i -й категории, ч.

Таблица 2 –Расчет затрат на основную заработную плату команды разработчиков

Категория исполнителя	Месячный оклад, р.	Часовой оклад, р.	Трудоемкость работ, ч	Итого, р.
Бизнес-аналитик	4 200,00	25,00	80	2 000,00
Системный архитектор	7 500,00	44,64	60	2 678,40
Программист	6 000,00	35,71	320	11 427,20
Тестирующий	3 800,00	22,62	120	2 714,40
Дизайнер	4 000,00	23,81	100	2 381,00
Итого				21 201,00
Премия и иные стимулирующие выплаты (50%)				10 600,50
Всего затрат на основную заработную плату разработчиков				31 801,50

Процесс вычисления включает в себя суммирование произведений часовых тарифных ставок на соответствующую трудоемкость работ для каждой роли в проекте. Для бизнес-аналитика этот показатель составил 2 000 р., для системного архитектора — 2 678,40 р., для программиста — 11 427,20 р., для тестирующего — 2 714,40 р. и для дизайнера — 2 381,00 р..

Суммарная величина выплат по базовым окладам для всей команды составила 21 201,00 р. После применения установленного коэффициента стимулирующих выплат итоговое значение основной заработной платы разработчиков составило 31 801,50 р. Данная сумма является базой для дальнейшего расчета отчислений в бюджет и оценки общей себестоимости программного продукта.

Для вычисления дополнительной заработной платы разработчиков используется показатель основной заработной платы, рассчитанный ранее, и норматив дополнительной заработной платы H_d . Значение H_d выбирается в диапазоне от 10% до 20%. Для данного проекта примем норматив в размере 10%, что является стандартным значением для учета выплат за непроработанное время (например, отпуска или выполнение государственных обязанностей). Подставляя в формулу 2 значение основной заработной платы команды, равное 31 801,50 р.

Таким образом, сумма дополнительной заработной платы разработчиков составляет 3 180,15 белорусских рублей.

Для расчета отчислений на социальные нужды воспользуемся формулой (3). Данный показатель отражает обязательные платежи в Фонд социальной защиты населения.

$$Z_d = \frac{Z_o \cdot H_d}{100} \quad (2)$$

где H_d – норматив дополнительной заработной платы;

Z_o – основная заработная плата разработчиков

Согласно методическим указаниям, норматив отчислений составляет 34,6%. Расчет производится от общей суммы фонда оплаты труда, которая включает в себя основную и дополнительную заработную плату разработчиков.

Используя полученные ранее значения, определим итоговую сумму, подставив значения в формулу 3.

$$P_{\text{соц}} = \frac{(Z_o + Z_d) \cdot H_{\text{соц}}}{100}, \quad (3)$$

где $H_{\text{соц}}$ – норматив отчислений в ФСЗН и Белгосстрах.

Таким образом, общая сумма отчислений на социальные нужды составляет 12 103,65 р. Эти средства являются обязательными издержками организации-разработчика и включаются в полную себестоимость программного продукта.

Следующим этапом в расчете затрат является определение прочих расходов по формуле 4. Данная статья затрат включает в себя общехозяйственные и общепроизводственные издержки, связанные с обеспечением процесса разработки.

Согласно методическим указаниям, норматив прочих расходов устанавливается в диапазоне от 30% до 40%. Для текущего расчета примем значение в размере 30%, исходя из специфики использования облачной инфраструктуры и удаленного характера работы команды. Расчет производится на основе ранее вычисленной основной заработной платы разработчиков.

$$P_{\text{пр}} = \frac{Z_o \cdot N_{\text{пр}}}{100}, \quad (4)$$

где $N_{\text{пр}}$ – норматив прочих расходов.

Таким образом, сумма прочих расходов составляет 9 540,45 р. На данном этапе у нас сформированы все необходимые компоненты для расчета полной себестоимости программного продукта.

Завершающим этапом расчета текущих затрат является определение расходов на реализацию по формуле 5. Данная статья включает затраты, связанные с коммерческой деятельностью, маркетингом и доведением программного продукта до конечного потребителя.

Согласно приведенным методическим указаниям, норматив расходов на реализацию устанавливается в пределах от 3% до 5%. С учетом выбранной ранее маркетинговой стратегии, включающей продвижение через магазины приложений и цифровой маркетинг, примем норматив в размере 5%. Расчет производится на базе основной заработной платы разработчиков.

$$P_{\text{пр}} = \frac{Z_o \cdot N_p}{100}, \quad (5)$$

где N_p – норматив расходов на реализацию.

Таким образом, расходы на реализацию составляют 1 590,08 р.

Произведем итоговый расчет общей суммы затрат на разработку и реализацию музыкального сервиса. Данный показатель объединяет в себе все ранее вычисленные статьи расходов. Подставим все полученные показатели из таблицы 1 в формулу 6.

$$Z_p = Z_o + Z_d + P_{\text{соц}} + P_{\text{пр}} + P_p \quad (6)$$

Таким образом, общая сумма затрат на разработку и реализацию информационной системы составляет 58 215,83 р. Эта величина представляет собой полную себестоимость продукта, на основе которой рассчитывается прогнозируемая прибыль и рентабельность проекта.

4.3 Расчет экономического эффекта от реализации программного средства на рынке

Экономический эффект организации-разработчика определяется как прирост чистой прибыли, остающейся в распоряжении предприятия после покрытия всех издержек и уплаты налогов[13].

На основе анализа востребованности нишевых музыкальных платформ и текущего состояния рынка независимой дистрибуции в СНГ, прогнозируется следующий объем реализации в течение первого расчетного года:

1 *B2B*-сегмент (корпоративные лицензии). Реализация 60 копий программного средства для региональных музыкальных лейблов и медиа-агрегаторов.

2 *B2C*-сегмент (подписки). Привлечение 10 000 активных пользователей, из которых, согласно рыночной статистике конверсии для *Freemium*-моделей (5%), 500 пользователей приобретут годовую подписку.

Ожидаемый объем продаж обоснован текущим дефицитом инструментов для автоматизированного анализа медиатек с использованием *track_features* (*BPM*, тональность) среди небольших правообладателей.

Цена реализации установлена на основе сравнительного анализа существующих решений и экспертной оценки трудозатрат на внедрение.

1 Корпоративная лицензия. Средняя стоимость аналогичных систем управления контентом (*CMS/DAM* для медиа) на рынке СНГ варьируется от 120 000 до 200 000 р. за внедрение. Для данной ИС установлена цена 150 000 бел. руб. за лицензию.

2 Индивидуальная подписка. Стоимость аналогичных сервисов (*Yandex Music*, *Spotify*, Звук) составляет в среднем от 5 до 12 р. в месяц. Для проектируемой системы цена установлена на уровне 10 р. в месяц. (или 120 р. в год).

Для расчета используются данные о полной себестоимости разработки и планируемой выручке.

Произведем расчет прироста чистой. Этот показатель позволяет оценить реальную экономическую выгоду организации-разработчика после выполнения всех налоговых обязательств. Для этого нам необходимо учитывать НДС, указанный на формуле 7.

$$\text{НДС} = \frac{\text{Ц}_{\text{отп}} \cdot N \cdot \text{Н}_{\text{д.с}}}{100\% + \text{Н}_{\text{д.с}}} \quad (7)$$

где $\text{Н}_{\text{д.с}}$ – ставка налога на добавленную стоимость в соответствии с действующим законодательством, %;
 $\text{Ц}_{\text{отп}}$ – отпускная цена копии (лицензии) программного средства, р.;
 N – количество копий.

На первом этапе определяется сумма налога на добавленную стоимость (НДС), входящая в отпускную цену программного средства. При отпускной цене 150 000 р. и ставке налога 20% величина НДС для одной копии составит 25 000 р.

Теперь посчитаем прирост от одной копии, по формуле 8. Рентабельность считать равной 30%. Ставку налога на прибыль составит 20%.

$$\Delta \text{П}_\text{ч}^p = (\text{Ц}_{\text{отп}} \cdot N - \text{НДС}) \cdot \text{Р}_{\text{пр}} \cdot \left(1 - \frac{\text{Н}_\text{п}}{100}\right) \quad (8)$$

где $\text{Р}_{\text{пр}}$ – рентабельность продаж копий;
 $\text{Н}_\text{п}$ – ставка налога на прибыль согласно действующему законодательству, %.

Для этого из отпускной цены вычитается полученная сумма НДС (150 000 р. минус 25 000 р., что дает 125 000 р.), затем полученное значение умножается на установленный коэффициент рентабельности продаж (30% или 0,3), а результат корректируется с учетом уплаты налога на прибыль по ставке 20% (умножением на коэффициент 0,8).

В результате пошагового расчета получаем: 125 000 р. умножить на 0,3, что равняется 37 500 р., и далее полученная сумма умножается на 0,8. Итоговый прирост чистой прибыли составляет 30 000 р.

При реализации одной корпоративной лицензии программного средства прирост чистой прибыли организации-разработчика составит 30 000 р.

Данный результат свидетельствует о высокой экономической эффективности проекта, так как прибыль от реализации даже одной единицы товара позволяет покрыть значительную часть операционных издержек, а при масштабировании продаж до запланированных объемов обеспечит полную окупаемость разработки в кратчайшие сроки.

4.4 Расчет показателей экономической эффективности разработки и реализации программного средства на рынке

На основании произведенных вычислений можно сделать следующий вывод: сумма инвестиций (затрат) на разработку значительно меньше суммы прогнозируемого годового экономического эффекта. Даже при реализации всего двух корпоративных лицензий, суммарный прирост чистой прибыли уже полностью перекрывает затраты на разработку.

Для расчета ROI воспользуемся итоговыми данными, полученными на предыдущих этапах и формулой 9. Суммарный прирост чистой прибыли рассчитаем исходя из реализации 5 корпоративных лицензий за год, что составит 150 000 р., чтобы показать реалистичную эффективность.

$$ROI = \frac{\Delta\Pi_q^p - Z_p}{Z_p} \cdot 100\% \quad (8)$$

Полученное значение рентабельности в 157,66% свидетельствует о крайне высокой эффективности проекта.

4.5 Вывод по результатам расчета

Анализ полученных показателей подтверждает высокую экономическую эффективность разработки и реализации программного продукта. Общая сумма затрат на разработку составила 58 215,83 р., в то время как ожидаемая чистая прибыль от продажи всего одной корпоративной лицензии достигает 30 000 р.. Показатель рентабельности инвестиций (ROI) на уровне 157,66% значительно превышает доходность альтернативных финансовых инструментов, что делает проект инвестиционно привлекательным.

Несмотря на благоприятные прогнозы, выход на массовый рынок сопряжен с определенными рисками. К основным факторам неопределенности относятся возможность появления более технологичных конкурентов, изменения в законодательстве относительно авторского права на цифровой контент и потенциальное снижение платежеспособного спроса в нишевом сегменте. Для минимизации данных рисков стратегия развития включает гибкую модель монетизации и постоянное обновление функционала на основе анализа аудио-характеристик треков.

Приближенная оценка точки безубыточности показывает, что для полного покрытия всех затрат на разработку и реализацию программного

средства достаточно обеспечить минимальный объем продаж в количестве 2 корпоративных лицензий. Учитывая емкость целевого рынка, включающего независимые музыкальные лейблы и медиа-платформы, достижение данного порога представляется гарантированным в течение первых нескольких месяцев коммерческой эксплуатации продукта. Таким образом, проект характеризуется низким уровнем финансового риска и высокой скоростью возврата инвестиций.

ЗАКЛЮЧЕНИЕ

В ходе дипломного проектирования успешно спроектирована и программно реализована масштабируемая клиент-серверная информационная система потокового вещания и анализа мультимедийного контента. Архитектура разработанного программного продукта базируется на современных технологических решениях, включая фреймворк *Spring Boot* на языке *Java* для серверной логики, реляционную систему управления базами данных *PostgreSQL* и высокопроизводительное S3-совместимое объектное хранилище *MinIO*. В рамках реализации структуры системы выделены и обособлены ключевые функциональные модули, среди которых центральное место занимают подсистема асинхронного глобального поиска, модуль каталогизации и ведения пользовательских профилей, а также интеллектуальный сервис анализа акустических характеристик аудиофайлов.

Для исключения классической проблемы избыточности *SQL*-запросов и снижения нагрузки на дисковую подсистему базы данных на уровне объектно-реляционного отображения применены декларативные графы сущностей *Hibernate*, позволившие объединить выборку связанных медиаданных в один эффективный проход. На уровне сетевого взаимодействия и трансляции контента реализован алгоритм пошагового асинхронного стриминга байтовых чанков, основанный на спецификации протокола *HTTP Range Requests*. Данное решение позволило организовать непрерывное воспроизведение и произвольную перемотку аудиодорожек без необходимости полной загрузки медиафайлов в оперативную память сервера, что кардинально повысило общую отказоустойчивость рантайма.

Непосредственный экономический эффект выражается в значительном снижении совокупной стоимости владения платформой благодаря полному отказу от коммерческих лицензий в пользу свободно распространяемого программного обеспечения и минимизации затрат на аренду облачных вычислительных мощностей за счет жесткого контроля утилизации памяти. Использование асинхронной параллельной загрузки данных на клиенте ускорило вывод релевантной информации, исключив блокировку интерфейса и снизив время ожидания ответа до минимума.

Дипломная работа выполнена по стандарту предприятия 2024 года[14].

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] *Yandex Cloud*. Официальная документация [Электронный ресурс] – Режим доступа: https://yandex.cloud/en/docs?utm_referrer=https%3A%2F%2Fwww.google.com%2F
- [2] Положение «Об электронном образовательном ресурсе по учебной дисциплине» – Режим доступа : www.bsuir.by/m/12_100229_1_185877.pdf
- [3] Купер, А. Об интерфейсе. Основы проектирования взаимодействий / А. Купер, Р. Рейман, Д. Кронин. – Санкт-Петербург : Питер, 2022. – 720 с.
- [4] Мартин, Р. Чистая архитектура / Р. Мартин – Санкт-Петербург : Питер, 2021. – 352 с.
- [5] *KMWLLC* [Электронный ресурс]. – Режим доступа : <https://kmwllc.com/index.php/2020/03/20/understanding-tf-idf-and-bm-25/>
- [6] Лузанов, П. В. Базы данных и *PostgreSQL*. Разработка приложений / П. В. Лузанов, Е. В. Рогов, И. В. Лёвшин. – Москва : *Postgres Professional*, 2021. — 456 с.
- [7] Нильсен, Я. Веб-дизайн: удобство использования веб-сайтов / Я. Нильсен. – Москва : Вильямс, 2000. – 512 с.
- [8] Иттен, И. Искусство цвета / И. Иттен ; пер. с нем. Л. Монаховой. – Москва : Д. Аронов, 2018. – 96 с.
- [9] Томас, К. *Expert Spring MVC and Web Flow* / К. Томас, С. Девулапалли, Д. Лонг. – Нью-Йорк : *Apress*, 2011. – 516 с.
- [10] Коспил, Д. *Spring 6 Recipes: A Problem-Solution Approach to Spring Framework* / Д. Коспил, М. Сервин. – Нью-Йорк : *Apress*, 2024. – 950 с.
- [11] Фаулер, М. *UML*. Основы. Краткое руководство по стандартному языку объектного моделирования / М. Фаулер. – Санкт-Петербург : Символ-Плюс, 2018. – 192 с.
- [12] Скрипкин, К. Г. Экономическая эффективность информационных систем / К. Г. Скрипкин. – Москва : ДМК Пресс, 2019. – 256 с.
- [13] Исаев, Д. В. Экономическое обоснование и оценка эффективности ИТ-проектов / Д. В. Исаев. – Москва : Издательский дом Высшей школы экономики, 2021. – 184 с.
- [14] СТП БГУИР 01-2024. Стандарт предприятия. Дипломное проектирование. Общие требования к оформлению пояснительной записки – Минск : БГУИР, 2024. – 179 с.

ПРИЛОЖЕНИЕ А (ОБЯЗАТЕЛЬНОЕ) ЛИСТИНГ КОДА

```
@GetMapping("/{id}/stream")
public ResponseEntity<StreamingResponseBody> streamTrack(
    @PathVariable Integer id,
    @RequestHeader HttpHeaders headers) {
    try {
        StorageService.S3ObjectWrapper s3Object = trackService.getTrackObject(id);
        InputStream inputStream = s3Object.getInputStream();
        long fileLength = s3Object.getContentLength();
        List<HttpRange> ranges = headers.getRange();

        if (ranges.isEmpty()) {
            StreamingResponseBody responseBody = outputStream -> {
                byte[] buffer = new byte[4096];
                int bytesRead;
                try (inputStream) {
                    while ((bytesRead = inputStream.read(buffer)) != -1) {
                        outputStream.write(buffer, 0, bytesRead);
                    }
                }
            };

            return ResponseEntity.status(HttpStatus.OK)
                .header(HttpHeaders.ACCEPT_RANGES, "bytes")
                .header(HttpHeaders.CONTENT_LENGTH, String.valueOf(fileLength))
                .contentType(MediaType.parseMediaType("audio/mpeg"))
                .body(responseBody);
        }

        HttpRange range = ranges.get(0);
        long start = range.getRangeStart(fileLength);
        long end = range.getRangeEnd(fileLength);
        long rangeLength = end - start + 1;
        long skipped = inputStream.skip(start);
        StreamingResponseBody responseBody = outputStream -> {
            byte[] buffer = new byte[4096];
            long bytesToRead = rangeLength;
            try (inputStream) {
                while (bytesToRead > 0) {
                    int maxRead = (int) Math.min(buffer.length, bytesToRead);
                    int bytesRead = inputStream.read(buffer, 0, maxRead);
                    if (bytesRead == -1) break;
                    outputStream.write(buffer, 0, bytesRead);
                }
            }
        };
    }
}
```

```

        bytesToRead -= bytesRead;
    }
}
};
return ResponseEntity.status(HttpStatus.PARTIAL_CONTENT) // 206
    .header(HttpHeaders.ACCEPT_RANGES, "bytes")
    .header(HttpHeaders.CONTENT_RANGE, "bytes " + start + "-" + end + "/" +
fileLength)
    .header(HttpHeaders.CONTENT_LENGTH, String.valueOf(rangeLength))
    .contentType(MediaType.parseMediaType("audio/mpeg"))
    .body(responseBody);
} catch (Exception e) {
    return ResponseEntity.status(HttpStatus.NOT_FOUND).build();
}
}
}
@RestController
@RequestMapping("/api/albums")
@CrossOrigin(origins = "*")
public class AlbumController {

    private final AlbumService albumService;
    @Autowired
    private AlbumRepository albumRepository;

    @Autowired
    private ArtistRepository artistRepository;

    @Autowired
    public AlbumController(AlbumService albumService) {
        this.albumService = albumService;
    }

    @PostMapping
    public ResponseEntity<?> createAlbum(
        @RequestParam("artistId") Integer artistId, // Ловим ?artistId= из URL
        @RequestBody Album album // Ловим JSON альбома (title, coverUrl, releaseDate)
    ) {
        Artist artist = artistRepository.findById(artistId).orElse(null);
        if (artist == null) {
            return ResponseEntity.badRequest().body("Ошибка: Артист с ID " + artistId + " не
найден!");
        }
        album.setArtist(artist);

        Album savedAlbum = albumRepository.save(album);

```

```

        return ResponseEntity.ok(savedAlbum);
    }

    @GetMapping("/{id}")
    public ResponseEntity<AlbumFullResponseDTO> getAlbum(@PathVariable Integer id) {
        try {
            AlbumFullResponseDTO albumInfo = albumService.getAlbumFullInfo(id);
            return ResponseEntity.ok(albumInfo);
        } catch (Exception e) {
            return ResponseEntity.status(HttpStatus.NOT_FOUND).build();
        }
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteAlbum(@PathVariable Integer id) {
        try {
            albumService.deleteAlbum(id);
            return ResponseEntity.noContent().build();
        } catch (Exception e) {
            return ResponseEntity.status(HttpStatus.NOT_FOUND).build();
        }
    }

    @GetMapping
    public ResponseEntity<List<Album>> getAllAlbums() {
        try {
            List<Album> albums = albumRepository.findAll();
            return ResponseEntity.ok(albums);
        } catch (Exception e) {
            return ResponseEntity.internalServerError().build();
        }
    }

    @GetMapping("/search")
    public ResponseEntity<List<Album>> searchAlbums(@RequestParam("query") String
query) {
        return ResponseEntity.ok(albumRepository.findByTitleContainingIgnoreCase(query));
    }
}

    @PostMapping("/upload")
    public ResponseEntity<?> uploadTrack(
        @RequestParam("title") String title,
        @RequestParam(value = "bpm", required = false) Integer bpm,
        @RequestParam(value = "duration", required = false) Integer duration,
        @RequestParam("artistId") Integer artistId,
        @RequestParam(value = "albumId", required = false) Integer albumId,
        @RequestParam("file") MultipartFile file) {

```

```

    try {
        if (file.isEmpty()) {
            return ResponseEntity.badRequest().body("Файл не выбран или пуст");
        }

        trackService.createTrack(title, bpm, duration, artistId, albumId, file);

        return ResponseEntity.ok().build();
    } catch (Exception e) {
        e.printStackTrace();
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body("Ошибка при загрузке трека на сервер: " + e.getMessage());
    }
}

@GetMapping("/{id}")
public ResponseEntity<TrackResponseDTO> getTrackById(@PathVariable Integer id) {
    try {
        Track track = trackService.getTrackById(id);

        TrackResponseDTO dto = new TrackResponseDTO(
            track.getTrackId(),
            track.getTitle(),
            track.getDuration(),
            track.getFilePath(),
            track.getBpm(),
            track.getCreatedAt()
        );

        return ResponseEntity.ok(dto);
    } catch (Exception e) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND).build();
    }
}

@Entity
@Table(name = "albums")
@Data
@JsonIgnoreProperties({"hibernateLazyInitializer", "handler"})
public class Album {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "album_id")

```

```

private Integer albumId;

@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "artist_id")
@JsonBackReference("artist-album")
private Artist artist;

@Column(nullable = false, length = 255)
private String title;

@Column(name = "release_date")
private LocalDate releaseDate;

@Column(name = "cover_url", length = 500)
private String coverUrl;

@OneToMany(mappedBy = "album", cascade = CascadeType.ALL, fetch = FetchType.LAZY)
@ToString.Exclude
@JsonManagedReference("album-track")
private List<Track> tracks;
}

@Entity
@Table(name = "artists")
@Data
public class Artist {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "artist_id")
    private Integer artistId;

    @OneToOne
    @JoinColumn(name = "user_id", nullable = false)
    @JsonBackReference("user-artist") // Обрежет обратный ход на User
    private User user;

    @Column(nullable = false, length = 255)
    private String name;

    @Column(columnDefinition = "TEXT")
    private String bio;

    @Column(name = "image_url", length = 500)
    private String imageUrl;

```

```

    @OneToMany(mappedBy = "artist", cascade = CascadeType.ALL, fetch = FetchType.LAZY)
    @ToString.Exclude
    @JsonManagedReference("artist-album")
    private List<Album> albums;
}

@Entity
@Table(name = "playlists")
@Data
@JsonIgnoreProperties({"hibernateLazyInitializer", "handler"})
public class Playlist {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "playlist_id")
    private Integer playlistId;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id", nullable = false)
    @JsonBackReference
    private User user;

    @Column(nullable = false, length = 255)
    private String title;

    @Column(name = "is_public")
    private Boolean isPublic = true;

    @Column(name = "created_at", updatable = false)
    private LocalDateTime createdAt;

    @ManyToMany
    @JoinTable(
        name = "playlist_tracks",
        joinColumns = @JoinColumn(name = "playlist_id"),
        inverseJoinColumns = @JoinColumn(name = "track_id")
    )
    private List<Track> tracks;

    @PrePersist
    protected void onCreate() {
        this.createdAt = LocalDateTime.now();
    }
    @Column(name = "cover_url", length = 500)
    private String coverUrl;
}

```

```

@Entity
@Table(name = "tracks")
@Data
@JsonIgnoreProperties({"hibernateLazyInitializer", "handler"})
public class Track {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "track_id")
    private Integer trackId;

    @JsonBackReference("album-track")
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "album_id")
    private Album album;

    @JsonIgnoreProperties({"tracks", "albums", "hibernateLazyInitializer", "handler"}) //
    // Игнорируем список треков внутри артиста, чтобы не было зацикливания
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "artist_id", nullable = false) // Имя колонки в БД (artist_id) из твоей
    // структуры таблицы
    private Artist artist;

    @Column(nullable = false, length = 255)
    private String title;

    @Column(nullable = false)
    private Integer duration;

    @Column(name = "file_path", nullable = false, length = 500)
    private String filePath; // Это наш S3 Key или URI файла

    @Column(nullable = false)
    private Integer bpm;

    @Column(name = "created_at", updatable = false)
    private LocalDateTime createdAt;

    @JsonIgnore
    @OneToOne(mappedBy = "track", cascade = CascadeType.ALL, fetch = FetchType.LAZY)
    private TrackFeatures trackFeatures;

    @PrePersist

```

```

protected void onCreate() {
    this.createdAt = LocalDateTime.now();
}
}

@Entity
@Table(name = "users")
@Data
@JsonIgnoreProperties({"hibernateLazyInitializer", "handler"})
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "user_id")
    private Integer userId;

    @Column(nullable = false, unique = true, length = 255)
    private String email;

    @Column(name = "password_hash", nullable = false, length = 255)
    private String passwordHash;

    @Column(nullable = false, length = 100)
    private String username;

    @Column(name = "created_at", updatable = false)
    private LocalDateTime createdAt;

    @PrePersist
    protected void onCreate() {
        this.createdAt = LocalDateTime.now();
    }

    @ManyToMany(fetch = FetchType.LAZY)
    @JoinTable(
        name = "user_saved_albums", // Имя связующей таблицы в БД
        joinColumns = @JoinColumn(name = "user_id"),
        inverseJoinColumns = @JoinColumn(name = "album_id")
    )
    private List<Album> savedAlbums;

    @OneToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "artist_id") // или mappedBy, смотря где лежит внешний ключ
    @JsonManagedReference("user-artist")
    private Artist artist;

    @Column(name = "is_artist", nullable = false)

```



```

private Boolean isArtist = false;
@OneToMany(mappedBy = "user", cascade = CascadeType.ALL, fetch = FetchType.LAZY)
private List<Playlist> playlists;
public Boolean isArtist(){
    return isArtist;
}

}

@Service
public class ArtistService {

    private final ArtistRepository artistRepository;
    private final AlbumRepository albumRepository;
    private final UserRepository userRepository;

    public ArtistService(ArtistRepository artistRepository,
                        AlbumRepository albumRepository,
                        UserRepository userRepository) {
        this.artistRepository = artistRepository;
        this.albumRepository = albumRepository;
        this.userRepository = userRepository;
    }

    @Transactional
    public Artist updateProfile(Integer userId, String name, String bio, String imageUrl) {
        User user = userRepository.findById(userId)
            .orElseThrow(() -> new RuntimeException("Пользователь не найден"));

        Artist artist = artistRepository.findByUser(user)
            .orElseThrow(() -> new RuntimeException("У вас нет профиля артиста"));

        if (name != null && !name.trim().isEmpty()) {
            artist.setName(name);
        }
        artist.setBio(bio);
        if (imageUrl != null) {
            artist.setImageUrl(imageUrl);
        }

        return artistRepository.save(artist);
    }

    @Transactional
    public Album createAlbum(Integer userId, String title, String coverUrl) {
        User user = userRepository.findById(userId)
            .orElseThrow(() -> new RuntimeException("Пользователь не найден"));
    }

```

```

        Artist artist = artistRepository.findByUser(user)
            .orElseThrow(() -> new RuntimeException("Только пользователи со статусом
артиста могут создавать альбомы"));

```

```

        Album album = new Album();
        album.setTitle(title);
        album.setCoverUrl(coverUrl);
        album.setArtist(artist); // Привязываем альбом строго к сущности Artist

        return albumRepository.save(album);
    }

```

```

    public Artist getProfileById(Integer userId) {
        User user = userRepository.findById(userId)
            .orElseThrow(() -> new RuntimeException("Пользователь не найден"));
        return artistRepository.findByUser(user)
            .orElseThrow(() -> new RuntimeException("Профиль артиста не найден"));
    }

```

@Service

```

public class AudioAnalysisService {

```

```

    public TrackFeatures analyze(MultipartFile multipartFile, Track track) {
        TrackFeatures features = new TrackFeatures();
        features.setTrack(track);
        features.setTrackId(track.getId());

```

```

        File tempFile = null;
        try {
            tempFile = convertMultipartFileToFile(multipartFile);
            AudioFile f = AudioFileIO.read(tempFile);
            Tag tag = f.getTag();
            String bpmStr = tag.getFirst(FieldKey.BPM);
            features.setBpm(bpmStr.isEmpty() ? 0 : Integer.parseInt(bpmStr));
            features.setGenre(tag.getFirst(FieldKey.GENRE));

```

```

            int duration = f.getAudioHeader().getTrackLength();
            track.setDuration(duration); // Обновляем основную сущность трека

```

```

            features.setEnergyLevel(0.7);
            features.setDanceability(0.5);
            features.setAcousticness(0.3);

```

```

        } catch (Exception e) {
            System.err.println("Ошибка анализа аудио: " + e.getMessage());

```

```

        features.setBpm(0);
        features.setGenre("Unknown");
    } finally {
        if (tempFile != null && tempFile.exists()) {
            tempFile.delete(); // Удаляем временный файл
        }
    }

    return features;
}

private File convertMultipartToFile(MultipartFile file) throws IOException {
    File convFile = new File(System.getProperty("java.io.tmpdir") + "/" +
file.getOriginalFilename());
    try (FileOutputStream fos = new FileOutputStream(convFile)) {
        fos.write(file.getBytes());
    }
    return convFile;
}

@Service
@RequiredArgsConstructor
public class PlaylistService {

    private final PlaylistRepository playlistRepository;
    private final UserRepository userRepository; // Нужен для привязки плейлиста к юзеру
    private final TrackRepository trackRepository;
    @Transactional(readOnly = true)
    public List<Playlist> getPlaylistsByUserId(Integer userId) {
        return playlistRepository.findByUserUserId(userId);
    }

    @Transactional
    public Playlist createPlaylist(Integer userId, String title) {
        User user = userRepository.findById(userId)
            .orElseThrow(() -> new RuntimeException("Пользователь не найден с id: " +
userId));

        Playlist playlist = new Playlist();
        playlist.setUser(user);
        playlist.setTitle(title);
        playlist.setIsPublic(true);

        return playlistRepository.save(playlist);
    }
}

```

```

@Transactional
public void addTrackToPlaylist(Integer playlistId, Integer trackId) {
    Playlist playlist = playlistRepository.findById(playlistId)
        .orElseThrow(() -> new RuntimeException("Плейлист не найден"));

    Track track = trackRepository.findById(trackId)
        .orElseThrow(() -> new RuntimeException("Трек не найден"));

    playlist.getTracks().add(track);

    playlistRepository.save(playlist);
}

@Transactional
public Playlist updatePlaylist(Integer playlistId, String title, String coverUrl) {
    Playlist playlist = playlistRepository.findById(playlistId)
        .orElseThrow(() -> new RuntimeException("Плейлист не найден"));
    playlist.setTitle(title);
    playlist.setCoverUrl(coverUrl);
    return playlistRepository.save(playlist);
}

@Transactional
public void deletePlaylist(Integer playlistId) {
    if (!playlistRepository.existsById(playlistId)) {
        throw new RuntimeException("Плейлист не найден");
    }
    playlistRepository.deleteById(playlistId);
}

@Transactional
public void removeTrackFromPlaylist(Integer playlistId, Integer trackId) {
    Playlist playlist = playlistRepository.findById(playlistId)
        .orElseThrow(() -> new RuntimeException("Плейлист не найден"));

    playlist.getTracks().removeIf(track -> track.getTrackId().equals(trackId));

    playlistRepository.save(playlist);
}

@Service
@RequiredArgsConstructor
public class StorageService {

    private final S3Client s3Client;

```

```

@Value("${aws.s3.bucket-name}")
private String bucketName;

public String uploadFile(MultipartFile file) throws IOException {
    String fileName = UUID.randomUUID() + "_" + file.getOriginalFilename();

    PutObjectRequest putObjectRequest = PutObjectRequest.builder()
        .bucket(bucketName)
        .key(fileName)
        .contentType(file.getContentType())
        .build();

    s3Client.putObject(putObjectRequest,
        RequestBody.fromInputStream(file.getInputStream(), file.getSize()));

    return fileName;
}

public InputStream getFileStream(String fileName) {
    try {
        GetObjectRequest getObjectRequest = GetObjectRequest.builder()
            .bucket(bucketName)
            .key(fileName)
            .build();

        // Возвращаем поток данных прямо из MinIO
        return s3Client.getObject(getObjectRequest);
    } catch (S3Exception e) {
        throw new RuntimeException("Ошибка при скачивании файла из MinIO: " +
            e.awsErrorDetails().errorMessage());
    }
}

public static class S3ObjectWrapper {
    private final InputStream inputStream;
    private final long contentLength;

    public S3ObjectWrapper(InputStream inputStream, long contentLength) {
        this.inputStream = inputStream;
        this.contentLength = contentLength;
    }

    public InputStream getInputStream() { return inputStream; }
}

```

```

        public long getLength() { return contentLength; }
    }

    public S3ObjectWrapper getFileObject(String fileName) {
        try {
            GetObjectRequest getObjectRequest = GetObjectRequest.builder()
                .bucket(bucketName)
                .key(fileName)
                .build();

            ResponseInputStream<GetObjectResponse> responseStream =
s3Client.getObject(getObjectRequest);
            long length = responseStream.response().contentLength();

            return new S3ObjectWrapper(responseStream, length);
        } catch (Exception e) {
            throw new RuntimeException("Ошибка чтения из MinIO: " + e.getMessage());
        }
    }
}

@Service
@RequiredArgsConstructor
public class TrackService {

    private final StorageService storageService;
    private final TrackRepository trackRepository;
    private final ArtistService artistService;
    private final ArtistRepository artistRepository;
    private final UserRepository userRepository;
    private final AudioAnalysisService audioAnalysisService;
    private final AlbumRepository albumRepository; // Добавляем

    @Autowired
    public TrackService(TrackRepository trackRepository,
        StorageService storageService, ArtistService artistService, ArtistRepository
artistRepository, UserRepository userRepository,
        AudioAnalysisService audioAnalysisService,
        AlbumRepository albumRepository) {
        this.trackRepository = trackRepository;
        this.storageService = storageService;
        this.artistService = artistService;
        this.artistRepository = artistRepository;
        this.userRepository = userRepository;
        this.audioAnalysisService = audioAnalysisService;
        this.albumRepository = albumRepository;
    }
}

```

```
}
```

```
public List<Track> searchTracks(String query) {  
    return trackRepository.findByTitleContainingIgnoreCase(query);  
}
```

```
@Transactional
```

```
public Track createTrack(String title, Integer bpm, Integer duration, Integer userId, Integer  
albumId, MultipartFile file) throws IOException {
```

```
    User user = userRepository.findById(userId)  
        .orElseThrow(() -> new RuntimeException("Пользователь с ID " + userId + " не  
найден"));
```

```
    Artist artist = user.getArtist();  
    if (artist == null) {  
        throw new RuntimeException("Этот пользователь не является артистом!");  
    }
```

```
    Album album = null;  
    if (albumId != null) {  
        album = albumRepository.findById(albumId)  
            .orElseThrow(() -> new RuntimeException("Альбом с ID " + albumId + " не  
найден"));  
    }
```

```
    String filePath = storageService.uploadFile(file);  
    Track track = new Track();  
    track.setTitle(title);  
    track.setFilePath(filePath);  
    track.setAlbum(album);  
    track.setArtist(artist); // Передаем корректного артиста (у которого ID = 3)  
    track.setBpm(bpm != null ? bpm : 0);  
    track.setDuration(duration != null ? duration : 0);
```

```
    Track savedTrack = trackRepository.save(track);  
    audioAnalysisService.analyze(file, savedTrack);
```

```
    return savedTrack;  
}
```

```
public StorageService.S3ObjectWrapper getTrackObject(Integer trackId) {  
    Track track = trackRepository.findById(trackId)  
        .orElseThrow(() -> new RuntimeException("Трек не найден"));
```

```

        return storageService.getFileObject(track.getFilePath());
    }
    public Page<Track> getAllTracks(Pageable pageable) {
        return trackRepository.findAll(pageable);
    }

    public Track getTrackById(Integer id) {
        return trackRepository.findById(id)
            .orElseThrow(() -> new RuntimeException("Трек с ID " + id + " не найден"));
    }
}

```

```

@Service
@RequiredArgsConstructor
public class StorageService {

```

```

    private final S3Client s3Client;

```

```

    @Value("${aws.s3.bucket-name}")
    private String bucketName;

```

```

    public String uploadFile(MultipartFile file) throws IOException {
        String fileName = UUID.randomUUID() + "_" + file.getOriginalFilename();

```

```

        PutObjectRequest putObjectRequest = PutObjectRequest.builder()
            .bucket(bucketName)
            .key(fileName)
            .contentType(file.getContentType())
            .build();

```

```

        s3Client.putObject(putObjectRequest,
            RequestBody.fromInputStream(file.getInputStream(), file.getSize()));

```

```

        return fileName;
    }
    public InputStream getFileStream(String fileName) {

```

```

        try {
            GetObjectRequest getObjectRequest = GetObjectRequest.builder()
                .bucket(bucketName)
                .key(fileName)
                .build();

```



```

        // Возвращаем поток данных прямо из MinIO
        return s3Client.getObject(getObjectRequest);
    } catch (S3Exception e) {
        throw new RuntimeException("Ошибка при скачивании файла из MinIO: " +
e.awsErrorDetails().errorMessage());
    }
}

public static class S3ObjectWrapper {
    private final InputStream inputStream;
    private final long contentLength;
    public S3ObjectWrapper(InputStream inputStream, long contentLength) {
        this.inputStream = inputStream;
        this.contentLength = contentLength;
    }
    public InputStream getInputStream() { return inputStream; }
    public long getContentLength() { return contentLength; }
}

public S3ObjectWrapper getFileObject(String fileName) {
    try {
        GetObjectRequest getObjectRequest = GetObjectRequest.builder()
            .bucket(bucketName)
            .key(fileName)
            .build();

        ResponseInputStream<GetObjectResponse> responseStream =
s3Client.getObject(getObjectRequest);
        long length = responseStream.response().contentLength();
        return new S3ObjectWrapper(responseStream, length);
    } catch (Exception e) {
        throw new RuntimeException("Ошибка чтения из MinIO: " + e.getMessage());
    }
}
}
}

```

```

<main class="main-layout">
    <div class="sidebar smooth-box">
        <div class="sidebar-section">
            <h3>Сохраненные альбомы</h3>
            <div id="sidebarAlbumsList"></div>
        </div>
        <div class="sidebar-section" style="border-top: 1px solid #252525; padding-top: 10px;">
            <h3>
                <span>Мои плейлисты</span>
                <button class="create-playlist-btn" onclick="createNewPlaylist()" title="Создать
плейлист">+</button>
            </h3>

```

```

        <div id="sidebarPlaylistsList"></div>
    </div>
</div>
<div class="content-area smooth-box">
    <div id="mainPageSection">
        <div id="recommendationsSubSection">
            <h2>Рекомендуемые альбомы</h2>
            <div class="recommendations-grid" id="recGrid"></div>
        </div>
        <div id="searchResultsSubSection" style="display: none;">
            <div class="search-filter-bar">
                <button class="filter-tab active" id="tab-ALL"
onclick="changeSearchFilter('ALL')">Все</button>
                <button class="filter-tab" id="tab-TRACK"
onclick="changeSearchFilter('TRACK')">Треки</button>
                <button class="filter-tab" id="tab-ALBUM"
onclick="changeSearchFilter('ALBUM')">Альбомы</button>
                <button class="filter-tab" id="tab-ARTIST"
onclick="changeSearchFilter('ARTIST')">Исполнители</button>
                <button class="filter-tab" id="tab-PLAYLIST"
onclick="changeSearchFilter('PLAYLIST')">Плейлисты</button>
            </div>
            <h2>Результаты поиска в единой таблице</h2>
            <table class="tracks-table">
                <thead>
                    <tr>
                        <th style="width: 50px; text-align: center;">Тун</th>
                        <th>Название / Имя</th>
                        <th>Дополнительно</th>
                        <th style="width: 60px; text-align: center;">Добавить</th>
                    </tr>
                </thead>
                <tbody id="unifiedSearchTableBody"></tbody>
            </table>
        </div>
    </div>
    <div id="detailsPageSection" style="display: none;">
        <div class="dynamic-header">
            <img class="dynamic-header-cover" id="detailsCover" src="">
            <div class="dynamic-header-info">
                <p id="detailsMetaLabel" style="text-transform: uppercase; letter-spacing: 1px;
color: var(--spotify-green); font-weight: bold; margin-bottom: 5px;"></p>
                <h1 id="detailsTitle">Название</h1>
            </div>
        </div>
        <table class="tracks-table">

```

```

<thead>
<tr>
  <th style="width: 40px; text-align: center;">#</th>
  <th>Название трека</th>
  <th>Характеристика (BPM)</th>
  <th style="width: 80px; text-align: right;">Длительность</th>
  <th style="width: 40px; text-align: center;">+</th>
</tr>
</thead>
<tbody id="detailsTracksList"></tbody>
</table>
</div>
<div id="studioPageSection" style="display: none;">
  <h2>Творческая Студия Музыканта</h2>
  <p style="color: var(--text-muted); margin-bottom: 25px;">Управление вашим
  публичным профилем, альбомами и медиафайлами в MinIO S3.</p>
  <div class="studio-container">
    <div class="studio-block">
      <h3>Профиль исполнителя</h3>
      <div class="studio-form">
        <label style="font-size: 12px; color: var(--text-muted);">Сценический
        псевдоним:</label>
        <input type="text" id="studioArtistName" placeholder="Ваше сценическое
        имя">
        <label style="font-size: 12px; color: var(--text-muted);">Биография
        артиста:</label>
        <textarea id="studioArtistBio" placeholder="Расскажите слушателям о себе
        и своем творчестве..."></textarea>
        <label style="font-size: 12px; color: var(--text-muted);">Ссылка на обложку
        профиля (URL):</label>
        <input type="text" id="studioArtistImage"
        placeholder="https://example.com/avatar.jpg">
        <button onclick="saveArtistProfileData()">Сохранить изменения</button>
      </div>
    </div>
    <div class="studio-block">
      <h3>Создать новый альбом</h3>
      <div class="studio-form">
        <label style="font-size: 12px; color: var(--text-muted);">Название
        релиза:</label>
        <input type="text" id="createAlbumTitle" placeholder="Например: Сборник
        хитов, EP 'Волна'">
        <label style="font-size: 12px; color: var(--text-muted);">Ссылка на обложку
        альбома (URL):</label>

```

```

        <input                type="text"                id="createAlbumCoverUrl"
placeholder="https://example.com/album-cover.jpg">
        <button                class="secondary-btn"
onclick="submitNewAlbumToServer()">Опубликовать альбом</button>
    </div>
</div>
<div class="studio-block" style="flex: 1.5;">
    <h3>Загрузить новый трек</h3>
    <div class="studio-form">
        <input type="text" id="uploadTrackTitle" placeholder="Название трека"
required>
        <input type="number" id="uploadTrackBpm" placeholder="BPM (Скорость)"
required>
        <input                type="number"                id="uploadTrackDuration"
placeholder="Длительность (в секундах)" required>
        <label style="font-size: 12px; color: var(--text-muted);">Привязать к
альбому:</label>
        <select id="uploadTrackAlbumSelect">
            <option value="">Выбрать альбом (Не обязательно)</option>
        </select>
        <label style="font-size: 12px; color: var(--text-muted); margin-top:
5px;">Выберите аудиофайл (.mp3):</label>
        <input type="file" id="uploadTrackFile" accept="audio/mpeg" required>
        <button onclick="submitTrackToStorage()">Загрузить трек в MinIO
S3</button>
    </div>
</div>
</div>
</div>
</div>
</main>

```

Обозначение					Наименование	Дополнительные сведения		
					Текстовые документы			
БГУИР ДП 1-53 01 02 039 ПЗ					Пояснительная записка	91 с.		
					Отчет о проверке на заимствования			
					Отзыв руководителя			
					Рецензия			
					Справка о внедрении результатов дипломного проекта			
					Графические документы			
ГУИР.0000000.001 ПД					Схема алгоритма работы рекомендаций	Формат А1		
ГУИР.0000000.002 ПЛ					Схема база данных музыкального сервиса	Формат А1		
ГУИР.0000000.003 ПЛ					Схема вариантов использования музыкального сервиса	Формат А1		

